

第三章

类和对象

从下面三个例子中，注意结构如何转换到类的发展过程

一、结构 🐼

(1) 一个简单的结构: Time / 将数据打包

```
struct Time{  
    int hour;  
    int minute;  
    int second;  
};
```

```
struct Time t1;  
t1.hour=10;  
t1.minute=24;  
t1.second=59;
```

```
struct Time *pt1;  
pt1=&t1;  
pt1->hour=10;  
pt1->minute=24;  
pt1->second=59;
```

一个例子:

```
int main(int argc, char* argv[]) {  
    char c;  
    struct Time{  
        int hour;  
        int minute;  
        int second;  
    };  
};
```

```
struct Time *pt1;  
pt1=&t1;  
(*pt1).hour=10;  
(*pt1).minute=24;  
(*pt1).second=59;
```

//结构的定义

```
Time timeObject, timeArray[10], *timePtr, *timePtr1, &timeRef=timeObject;
```

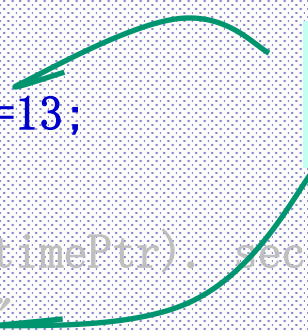
//结构的引用

```
timeObject.hour=3;    timeObject.minute=10;    timeObject.second=55;
```

```
cout<<"时间1为: "<<timeObject.hour<<": "  
    <<timeObject.minute<<": "<<timeObject. second<<endl;
```

```
timeArray[6]=timeObject;  timeArray[6].hour=23;  
cout<<"时间2为: "<<timeArray[6].hour<<": "  
    <<timeArray[6].minute<<": "<< timeArray[6].second<<endl;
```

```
timePtr=&timeObject;    (*timePtr).hour=13;  
cout<<"时间3为: "<<(*timePtr).hour  
    <<": "<<(*timePtr).minute<<": "<<(*timePtr). second<<endl;  
cout<<"时间3为: "<<timePtr->hour<<": "  
    <<timePtr->minute<<": "<<timePtr->second <<endl;
```

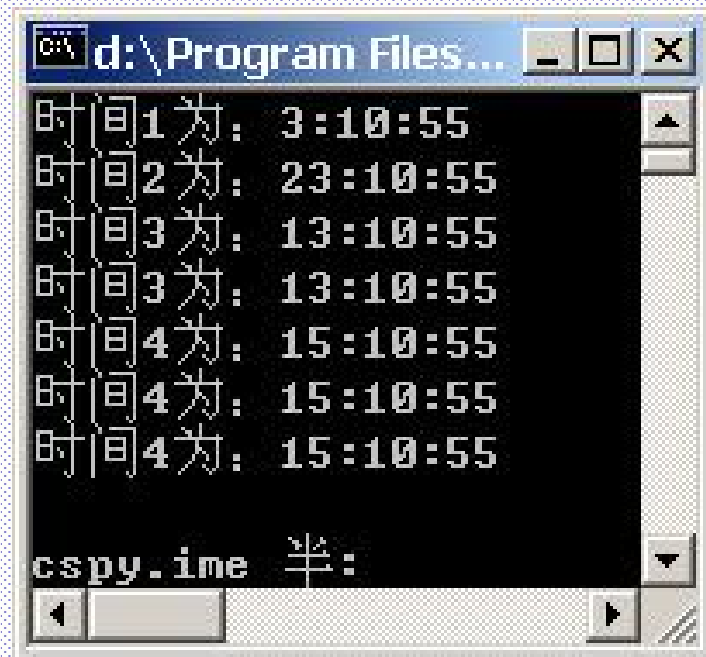


```
(*timePtr).hour=13;  
timePtr->hour=13;
```

```
timeRef.hour=15; //这时数据不是独立存在的，是从属
cout<<"时间4为："<<timeRef.hour<<":"<<timeRef.minute<<":"
<<timeRef.second <<endl;
```

```
cout<<"时间4为："<<timeObject.hour<<":"<<timeObject.minute<<":"
<<timeObject.second<<endl;
cout<<"时间4为："<<(*timePtr).hour<<":"
<<(*timePtr).minute<<":"<<(*timePtr).second<<endl;
```

```
}
```



```
d:\Program Files...
时间1为: 3:10:55
时间2为: 23:10:55
时间3为: 13:10:55
时间3为: 13:10:55
时间4为: 15:10:55
时间4为: 15:10:55
时间4为: 15:10:55
cspy.ime 半:
```

(2)、一个带有处理/操作/方法的结构:Time / 将数据与方法打包

例子二:

```
struct Time {    // structure definition
    int hour;    // 0-23
    int minute;  // 0-59
    int second;  // 0-59
};
```



数据与方法/函数是分离的

```
void printMilitary( const Time & ); // function prototype
void printStandard( const Time & ); // prototype
```

```
int main() {
    Time dinnerTime;    //variable of new type Time

    //set members to valid values
    dinnerTime.hour = 18;
    dinnerTime.minute = 30;
```

```
dinnerTime.second = 0;
```

```
cout << "Dinner will be held at ";
```

```
void printMilitary( const Time & ); // function prototype  
void printStandard( const Time & ); // prototype
```

```
printMilitary( dinnerTime );
```

//没有自己机器，借机房电脑玩游戏

//注意：对象作为参数；函数不附属于对象

```
cout << " military time,\nwhich is ";
```

```
printStandard( dinnerTime );
```

```
struct Time t1;  
t1.hour=10;  
t1.minute=24;  
t1.second=59;
```

```
cout << " standard time.\n";
```

```
// set members to invalid values
```

```
dinnerTime.hour = 29;
```

```
dinnerTime.minute = 73;
```

```
cout << "\nTime with invalid values: ";
```

```
printMilitary( dinnerTime );
```

```
cout << endl;
```

```
return 0;
```

```
}
```

```

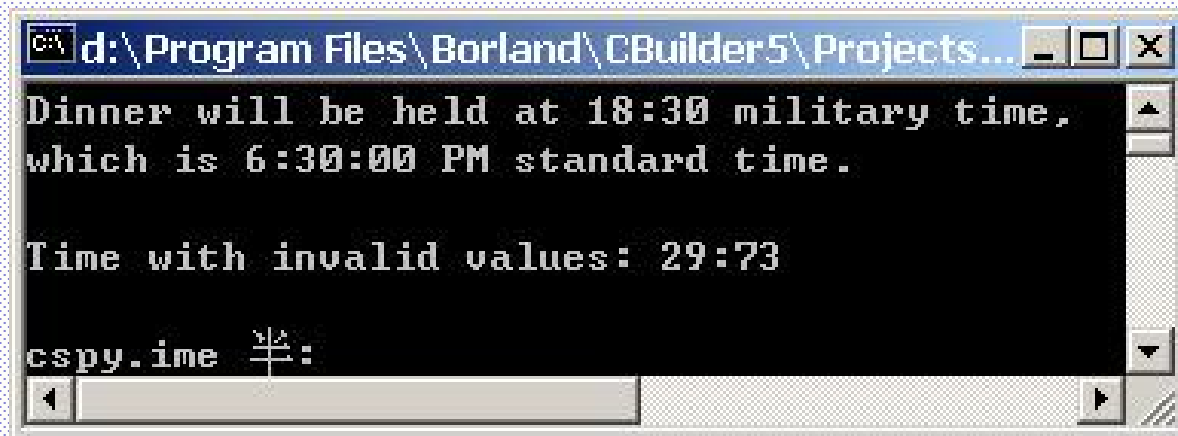
void printMilitary( const Time &t ) {
    cout << ( t.hour < 10 ? "0" : "" ) << t.hour << ":"
        << ( t.minute < 10 ? "0" : "" ) << t.minute;
}

```

```

void printStandard( const Time &t ) {
    cout << ( ( t.hour == 0 || t.hour == 12 ) ?
        12 : t.hour % 12 )
        << ":" << ( t.minute < 10 ? "0" : "" ) << t.minute
        << ":" << ( t.second < 10 ? "0" : "" ) << t.second
        << ( t.hour < 12 ? " AM" : " PM" );
}

```



The screenshot shows a console window titled "d:\Program Files\Borland\CBuilder5\Projects...". The output text is as follows:

```

Dinner will be held at 18:30 military time,
which is 6:30:00 PM standard time.

Time with invalid values: 29:73

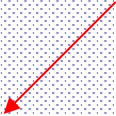
cspy.ime 半:

```

The window includes standard Windows-style window controls (minimize, maximize, close) and a scrollbar on the right side.

(3)、一个实现**数据与处理封装在一起的结构**:Time
例子三：例子二的改进：数据和方法的封装

方法和数据同属于结构



```
struct Time {
```

```
    int hour;
```

```
    int minute;
```

```
    int second;
```

```
void printMilitary() {
```

```
    cout << ( hour < 10 ? "0" : "" ) << hour << ":"  
          << ( minute < 10 ? "0" : "" ) << minute;
```

```
}
```

```
void printStandard() {
```

```
    cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )  
          << ":" << ( minute < 10 ? "0" : "" ) << minute  
          << ":" << ( second < 10 ? "0" : "" ) << second  
          << ( hour < 12 ? " AM" : " PM" );
```

```
}
```

```
}; //end of struct time
```

我有汽车（数据）
我会开车（操作）


```
int main() {
    Time dinnerTime;

    dinnerTime.hour = 18;
    dinnerTime.minute = 30;
    dinnerTime.second = 0;
    cout << "Dinner will be held at ";
    dinnerTime.printMilitary();
    cout << " military time, \nwhich is ";
    dinnerTime.printStandard();
    cout << " standard time. \n";
    dinnerTime.hour = 29;
    dinnerTime.minute = 73;
    cout << " \nTime with invalid values: ";
    dinnerTime.printMilitary();
    cout << endl;
    return 0;
}
```

//有车，自己开车

void printMilitary(const Time &); // function prototype
void printStandard(const Time &); // prototype

没有自己机器，借机房电脑玩游戏

//开车动作/未确定谁的车

printMilitary(dinnerTime);

//函数附属于对象，注意这里没有了参数

//我开车：开自己的车

//注意：在缺省情况下，结构中的成员是公有的。

(4)、访问结构成员

访问结构成员或(类成员)时，使用成员访问运算符：

包括圆点运算符(.)和箭头运算符(->)。

圆点运算符通过对象的变量名或对象的引用访问结构(或类成员)。

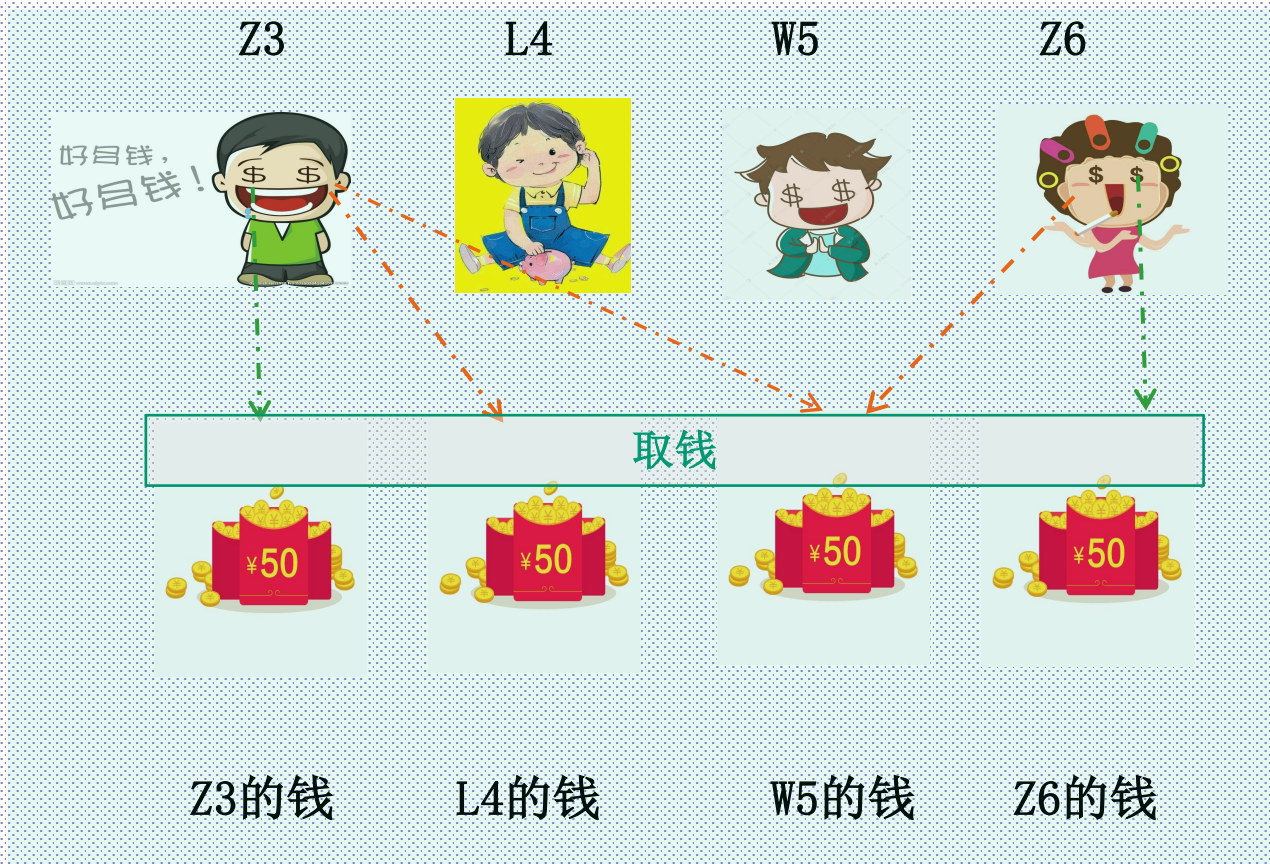
箭头运算符通过对象指针访问结构或类成员。

普通模块化C

->struct结构体公有

->class类私有封装

我的我的，都是我的 没有安全感



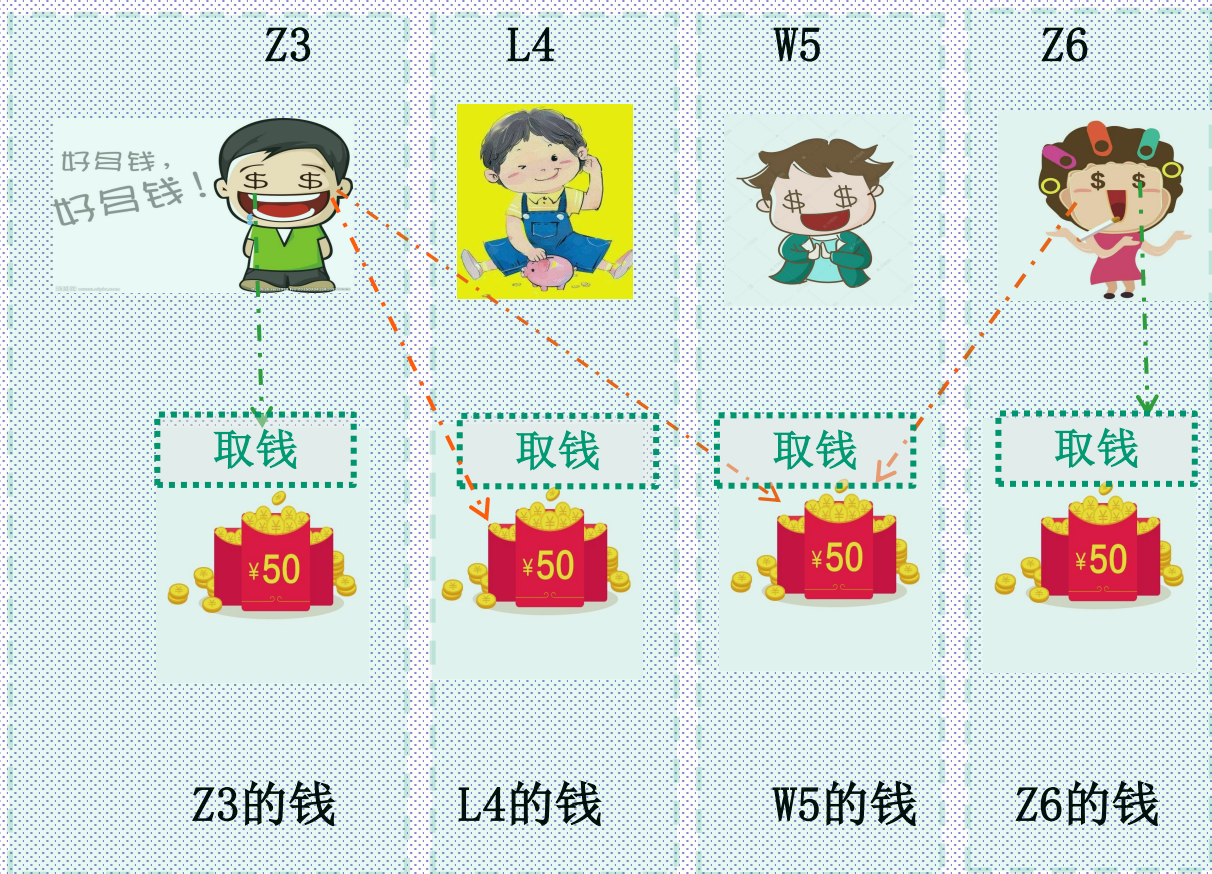
普通模块化C

->struct结构体/公有
防君子不防小人

->class类私有封装

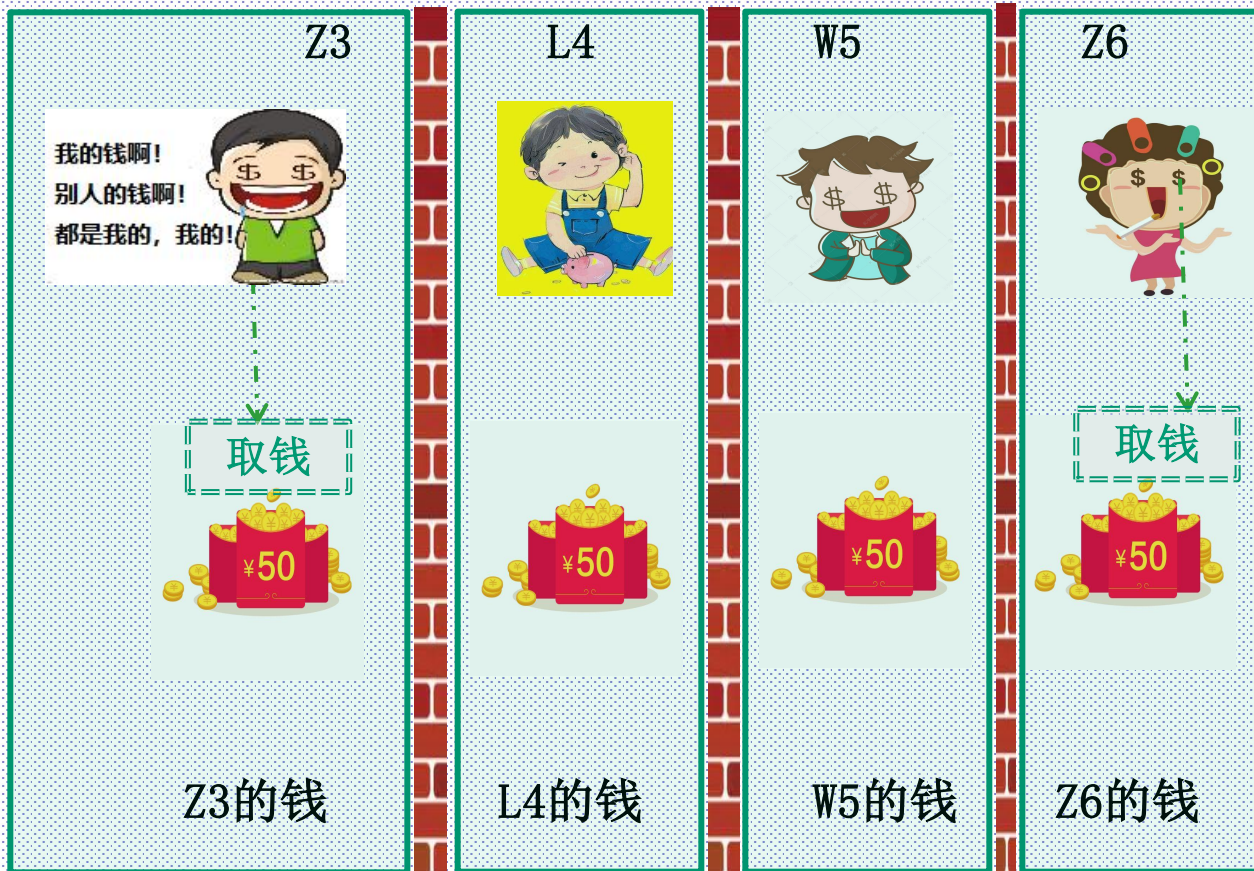
我的我的，都是我的

还是没有安全感；因为缺省公有



别人的拿不到了。。。

安全了!!



二、类

注意：OOP(面向对象的设计) **将数据(属性)和函数(行为)封装**到称为类的软件包中。一个类可以多次复用，建立多个类对象。

类具有信息隐藏特性，即类对象只知道如何通过定义良好的接口与其他类对象通行，而不需知道其他类的实现方法。信息隐藏是良好的软件工程的一个重要环节。

(1)、类的声明格式：

class 类名 {

[**private:**]

私有数据成员和成员函数

public:

公有数据成员和成员函数

protected:

保护数据成员和成员函数

};

[]表示缺省设置；缺省是私有的（不同于结构体）

私有的世界观，看见同学拿着一个新手机：

哇，你买了个新手机♡♡♡♡

而不是：

哇，你哪里捡的！！！！

在C++中，新的数据类型可以用class来构造，
class声明的语法与C语言中的struct声明相似，
只是class中还包含函数声明(实际上struct也可以包含函数)。

例：

```
class point {  
    int x, y;  
public:  
    void setpoint(int, int);  
};
```

在上例中声明一个名为Point的类：

它所包含的数据内容(属性)为x、y；

它所包含的函数操作为setpoint，此函数是对其数据内容x，y的操作。

私有数据：家里可以直接用/不给别人用
家：class point {}; 内部
缺省/默认/default: 私有

公有函数：可以有规则的借给别人用
这里的规则，就是这个公有函数
可以称之为[接口]

```
point p1;
```

```
p1.x=5;    // 出错；因为这肯定是在class point外面的代码
```

```
请使用setpoint() //公有接口
```

(2)、用类来实现Time结构

例一：

```
class Time {
```

```
    public:
```

```
        Time();           // constructor/构造函数：做初始的工作
```

```
        void setTime( int, int, int );    //注意，这里只有函数原型
```

```
        void printMilitary();
```

```
        void printStandard();
```

```
    private:
```

```
        int hour;         // 0 - 23
```

```
        int minute;      // 0 - 59
```

```
        int second;      // 0 - 59
```

```
};
```

①函数部分，类似C语言的函数定义

②数据部分，类似C语言的变量定义

类：有②自己的数据和①对数据的操作函数

如果一个类型变复杂了，就要做很多事情，有一个流程，一个手续
比如开通银行卡，要[填表，证件，密码]等

因此就有了构造函数（打包处理开通银行卡的一系列手续）

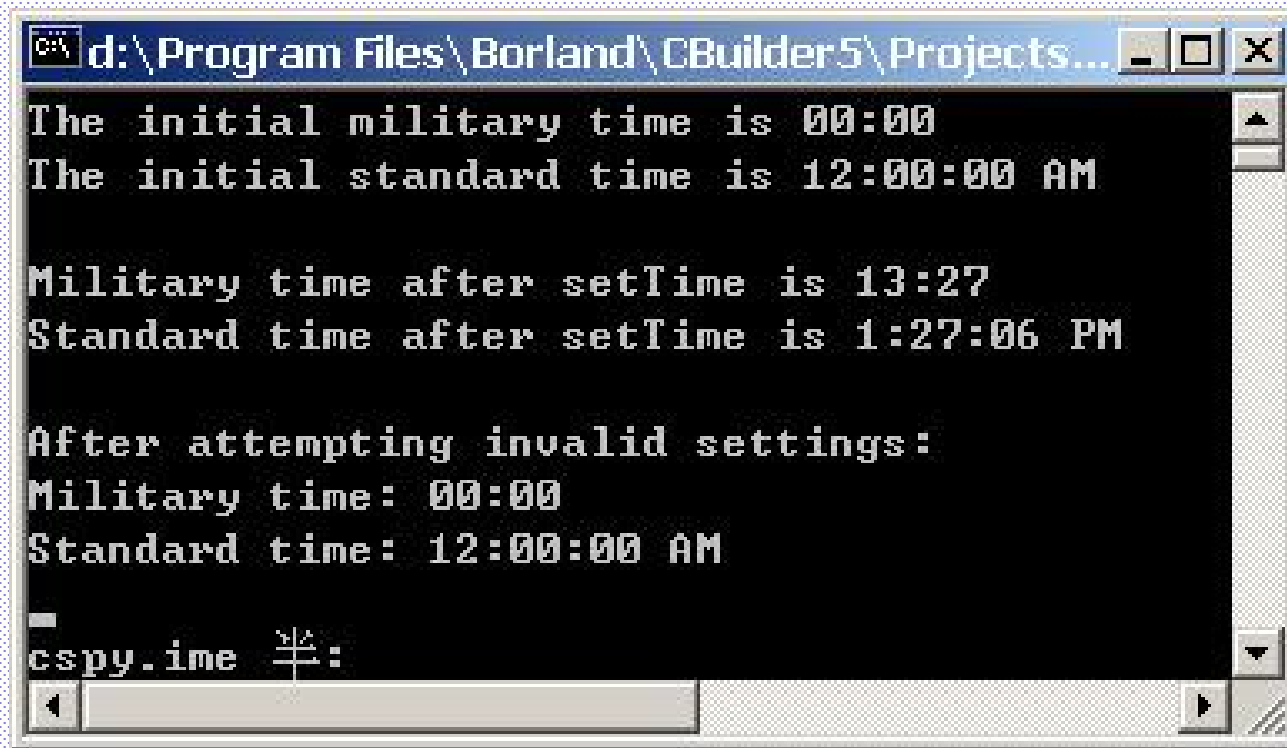
Time::Time() { hour = minute = second = 0; } //注意, 该函数与类同名

```
void Time::setTime( int h, int m, int s ) {  
    hour = ( h >= 0 && h < 24 ) ? h : 0;  
    minute = ( m >= 0 && m < 60 ) ? m : 0;  
    second = ( s >= 0 && s < 60 ) ? s : 0;  
}
```

```
void Time::printMilitary() {  
    cout << ( hour < 10 ? "0" : "" ) << hour << ":"      09:30  
        << ( minute < 10 ? "0" : "" ) << minute;        11:58  
}
```

```
void Time::printStandard() {  
    cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )  
        << ":" << ( minute < 10 ? "0" : "" ) << minute  
        << ":" << ( second < 10 ? "0" : "" ) << second  
        << ( hour < 12 ? " AM" : " PM" );  
}
```

```
int main() {  
    Time t; //调用构造函数进行初始化等 Time::Time() { hour = minute = second = 0; }  
           //int i=0; 普通变量初始化  
    cout << "The initial military time is ";    t.printMilitary();  
    cout << "\nThe initial standard time is ";    t.printStandard();  
  
    t.setTime( 13, 27, 6 ); //不直接设置, 通过对外的公有函数  
    cout << "\n\nMilitary time after setTime is "; t.printMilitary();  
    cout << "\nStandard time after setTime is ";    t.printStandard(); 对比  
  
    t.setTime( 99, 99, 99 );  
    cout << "\n\nAfter attempting invalid settings:"  
        << "\nMilitary time: ";  
    t.printMilitary();  
    cout << "\nStandard time: ";    t.printStandard();  
    cout << endl;  
    return 0;  
}
```



The screenshot shows a DOS-style command window with a title bar containing the file path "d:\Program Files\Borland\CBuilder5\Projects...". The window has standard minimize, maximize, and close buttons. The main area is black with white text. The text is as follows:

```
The initial military time is 00:00
The initial standard time is 12:00:00 AM

Military time after setTime is 13:27
Standard time after setTime is 1:27:06 PM

After attempting invalid settings:
Military time: 00:00
Standard time: 12:00:00 AM

cspy.ime  3/2:
```

At the bottom, there is a horizontal scrollbar and a small icon in the bottom right corner.

(3)、数据成员/函数成员、公有/私有/保护

数据成员可以是任何数据类型，除了register、extern

不能在类的声明中给类赋初值（区别函数缺省值/版本），只有在对象定义后才可以，why?

private:私有成员只能被该类的成员函数访问（I访问my）

public:公有成员可以被程序中的其他函数（包括main和其他类的函数）访问，它们是类的接口，通过接口为外部提供类操作的方法。是个窗口（接口/interface）

域：区别权力范围。 类、namespace、main函数、其他非类内函数

protected:保护成员可以被该类的派生类的成员函数访问，但对其它类是隐蔽的，不可访问的。

以Time为例，说明private与public

t.hour=5;

如何设置t的hour

```
class Time {  
    private:  
        int hour=0;    // 视版本允许  
        int minute=0;  //  
        int second=0;  //  
        ...  
};
```

新版规则和JAVA?

问题：引用和private数据成员/小心灵活的功能，除非你有足够的掌控力
如何破坏封装性？（或简单提醒）合理使用引用和指针

```
#include <iostream>

class Time {
public:
    Time( int = 0, int = 0, int = 0 );
    void setTime( int, int, int );
    int getHour();
    int &badSetHour( int );    // DANGEROUS reference return
private:
    int hour;
    int minute;
    int second;
};
```

```
Time::Time( int hr, int min, int sec ) { setTime( hr, min, sec ); }
```

```
void Time::setTime( int h, int m, int s ) {  
    hour    = ( h >= 0 && h < 24 ) ? h : 0;  
    minute  = ( m >= 0 && m < 60 ) ? m : 0;  
    second  = ( s >= 0 && s < 60 ) ? s : 0;  
}
```

```
int Time::getHour() { return hour; }
```

```
int &Time::badSetHour( int hh ) {  
    hour = ( hh >= 0 && hh < 24 ) ? hh : 0;  
    return hour;           // DANGEROUS reference return  
}
```

return hour;

我是私有的



int &hourRef = t.badSetHour(20);

main函数中（我是外部的）

int &Time::badSetHour()

```

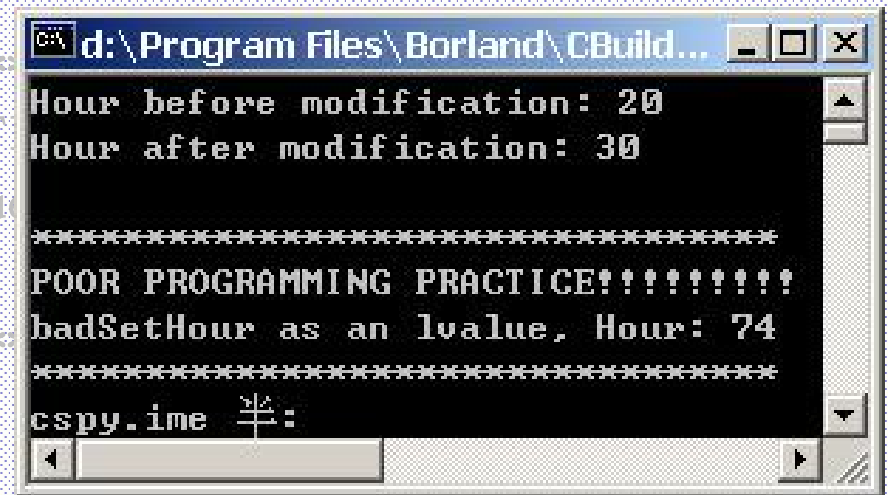
int main() {
    Time t;
    int &hourRef = t.badSetHour( 20 );

    cout << "Hour before modification: " << hourRef;
    hourRef = 30; // modification with invalid value
    cout << "\nHour after modification: " << t.getHour();

    // Dangerous: Function call that returns
    // a reference can be used as an lvalue!
    t.badSetHour(12) = 74;
    cout << "\n\n*****
    << "POOR PROGRAMMING PRACTICE!!!!!!!"
    << "badSetHour as an lvalue, Hour: 74"
    << t.getHour()
    << "\n*****";

    return 0;
}

```



```

d:\Program Files\Borland\CBuild...
Hour before modification: 20
Hour after modification: 30
*****
POOR PROGRAMMING PRACTICE!!!!!!
badSetHour as an lvalue, Hour: 74
*****
cspy.ime 半:

```

(4)、成员函数的定义

成员函数的声明有两种：

一种是在类声明中只给出函数原型，然后在类的外部定义：要使用到域运算符::。如上面的例子。

```
Time::Time( int hr, int min, int sec ) {  
  
void Time::setTime( int h, int m, int s )  
    hour    = ( h >= 0 && h < 24 ) ? h : 0
```

另一种定义在类的内部(包括显式与隐式)

①其中显式用inline嵌入到类内/形式上写在类外用::

②隐式直接写在类内

第二种的隐式： // 写在类内，直接看到代码

```
class Time {    // structure definition
    int hour;    // 0-23
    int minute;  // 0-59
    int second;  // 0-59

    void printMilitary()
    {
        cout << ( hour < 10 ? "0" : "" ) << hour << ":"
              << ( minute < 10 ? "0" : "" ) << minute;
    }

    void printStandard()
    {
        cout << ( ( hour == 0 || hour == 12 ) ?
                  12 : hour % 12 )
              << ":" << ( minute < 10 ? "0" : "" ) << minute
              << ":" << ( second < 10 ? "0" : "" ) << second
              << ( hour < 12 ? " AM" : " PM" );
    }
};
```

第二种显式： 在外部定义的方法前加inline关键词限定，从而在编译时将其转换为内联。//当然，也可以不设为inline；不需要太纠结inline

```
class Time {  
    public:  
        Time();  
        void setTime( int, int, int );  
    private:  
        int hour;      // 0 - 23  
        int minute;    // 0 - 59  
        int second;    // 0 - 59  
};
```

```
inline Time::Time() { hour = minute = second = 0; }
```

//下面一句是否等价? 构造函数无返回值

//inline void Time::Time() { hour = minute = second = 0; }

```
inline void Time::setTime( int h, int m, int s ) {  
    hour = ( h >= 0 && h < 24 ) ? h : 0;  
    minute = ( m >= 0 && m < 60 ) ? m : 0;  
    second = ( s >= 0 && s < 60 ) ? s : 0;  
}
```

(5)、类和对象的关系

类和对象之间的关系，可以用整型int和整型变量i之间的关系来类比。

C++把类的变量称为类的对象，对象是类的实例化/如同i是int的变量。
声明了一个类便声明了一个新的类型，类本身不接受和存储具体的值；
只作为生成具体对象的一种“样板”；

只有定义了对对象后，系统才为对象分配存储空间，才能存储数据。 `int=3 i=3`

如果在函数外，在声明类的同时定义的对象是一个全局的对象。如：

```
class point{  
    ...  
} op1, op2;
```

```
main() {  
    point p1, p2;  
}
```



“样板”：类

“实物”：对象



类也可以有样板：
类模板：制造机
器的机器



(5-2)、类/对象的内外关系

下面的代码，p1, p2在main里；对类point来说，main函数是外部的。
因此，p1, p2只能通过公有的成员函数间接访问私有成员

```
class point{  
    . . .  
};
```

```
main() {  
    point p1, p2;  
}
```

在java里，main函数被封装成一个类。这也是两者OO血统之争的一个点。
不过这样可以明显的看出，2者分属2个类

因此main所在类内，访问另一个类必受约束

外来访问

```
public class MainMethod{  
    public static void main(String[] args) {  
        for (int i = 0; i < args.length ; i++) {  
            System.out.println("第" + (i+1) + "个参数为: " + args[i]);  
        }  
    }  
}
```

```
class point{  
    . . .  
}
```

(6)、对象的定义和使用

定义方法1:

```
class point{  
    private:  
        int x,y;  
    public:  
        void setpoint(int, int) ;  
} op1, op2;
```

定义方法2:

```
class point{  
    private:  
        int x,y;  
    public:  
        void setpoint(int, int) ;  
};  
point p1, p2;
```

对象的使用

对象名.数据成员名

对象名.成员函数名(参数表) `t.setTime( 13, 27, 6 );`

对象指针名->数据成员名 `timePtr->hour`

对象指针名->成员函数名(参数表)

如:

```
point p1,*p2;    //假设成员数据x, y公有; 假设p2已指向某个对象
cout<<p1.x;
cout<<p2->y;
cout<<(*p2).y;
```

不能直接引用x, y (需要通过对象: `p1.x`) ; //因为数据从属于对象
对象是封装的, 它的成员都属于某个对象

对象赋值:

```
point p1,p2;
```

```
p2=p1;
```

等同于:

```
p2.x=p1.x;
```

```
p2.y=p1.y;
```

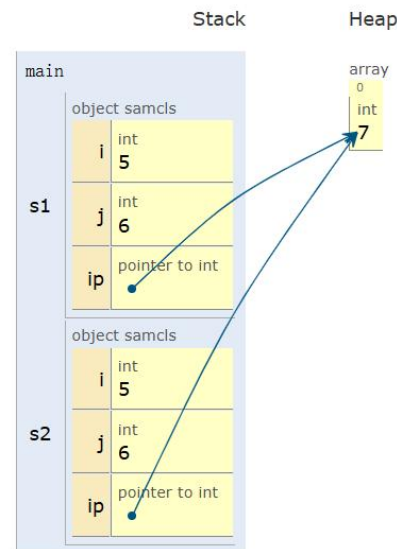
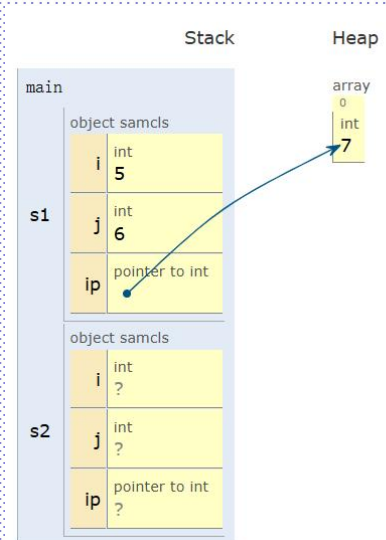
即将数据值复制过去，对象空间仍然是独立的。

如果类中存在指针，缺省复制可能会产生错误
Why?

```
class sample{  
    public:  
        int i,j;  
        int *ip;  
} s1,s2;
```

```
Student::Student(char *name1,char *stu_no1,int score1){  
    name=new char[strlen(name1)+1];stu_no=new char[strlen(stu_no1)+1];  
    strcpy(name,name1);strcpy(stu_no,stu_no1);  
    score=score1;}  
Student::Student(char *name1,char *stu_no1,int score1){  
    name=new char[strlen(name1)+1];stu_no=new char[strlen(stu_no1)+1];  
    strcpy(name,name1);strcpy(stu_no,stu_no1);  
    score=score1;}  
Student::~Student(){  
    delete name;delete stu_no;  
}
```

浅拷贝的
问题:
当s2=s1后



深拷贝:
先认识一下

类的作用域

一个类的成员函数可以不受限制地引用该类自己的数据成员，而在该类作用域之外，对该类的数据成员和成员函数的引用要受到一定的限制。

```
class abc{  
    public:  
        int i;  
        void iinc();  
        void jset(int);  
        void jprint();  
    private:  
        int j;  
        void jinc();  
};
```

1

```
int main() {  
    int i;    abc t1;  
    t1.i=5;    t1.iinc();  
    cout<<t1.i<<endl;  
    //下面两句是否正确  
    //t1.j=6;    t1.jinc();  
    t1.jset(11);    t1.jprint();  
}
```

2

```
void abc::iinc() { i++; }  
void abc::jinc() { j++; }  
void abc::jset(int sj) { j=sj; jinc(); }  
void abc::jprint() { cout<<"j="<<j<<endl; }
```

1

一个建议:

一般应将数据成员全部定义成`private;` //保护目的

而将对这些数据成员操作的成员函数设置为`public;` //对外开放

这样对数据的操作只能通过调用这些函数来完成(接口)。

为什麼? 如:

`private:`

`int x, y;`

`public:`

`void setX(int);` //比如银行取钱, 通过柜台工作人员或ATM, 有验证

`int getX();`

接口interface

成员初始化列表:

(1) 方便简洁

(2) **const**、引用等情况下，不能多次赋值，所以必须用初始化列表完成初始化

(3) 对象成员

P66, 例子3.7

```
class foo
{
    public:
        string name ;
        int id ;
        public:
        foo(string s, int i):name(s), id(i) // 初始化列表
        {   name=s; id=i;   } // 也可以在这里，二选一
};
```

成员初始化列表:

```
class ConstRef {
```

```
private:
```

```
    int i;
```

```
    const int ci;
```

```
    int &ri;
```

```
public:
```

```
    ConstRef(int ti, int tci, int tri);
```

```
};
```

```
ConstRef::ConstRef(int ti, int tci, int tri) :ci(tci), ri(tri) {
```

```
    i=ti;
```

```
}
```

```
int main() {
```

```
    ConstRef cr(3, 4, 5);
```

```
    //.....
```

```
    return 0;
```

```
}
```

构造对象的时候**必须马上初始化**；然后不能改变

引用或const
只能这里，
不能函数体括
号里

初始化顺序：将上页程序略作修改，以方便调试/下面是VC6/其他编译器/略过

```
#include <iostream>
using namespace std;

class ConstRef {
private:
    int i;
    const int ci;
    int ri;
public:
    ConstRef(int ti, int tci, int tri);
};
```

1: 声明处

成员变量在使用初始化列表初始化时，与构造函数中初始化成员列表的顺序无关，只与定义成员变量的顺序有关。

2-1: 定义处列表

```
ConstRef::ConstRef(int ti, int tci, int tri):ri(tri),ci(ri)
{
    i=ci;
    i=ri;
}
```

2-2: 定义处函数体;

2条语句分别对应下面2个图

```
int main() {
    ConstRef cr(3, 4, 5);
    return 0;
}
```

//.....

上下文: ConstRef::ConstRef(int, int, int)

名称	值
ci	-858993460
i	-858993460
this	0x0019ff24
i	-858993460
ci	-858993460
ri	5

this	0x0019ff24
i	5
ci	-858993460
ri	5

初始化的顺序，对比跟踪数据中对象的数据值：this

2-1的两个初始化实际上是按1的顺序执行的，即声明顺序优先。故ci脏数据，因为ri没有先初始化。

2-2的i的初始化，是迟于2-1的列表的。因为i获得了ci的脏数据，然后再获得ri的值

三、类的构造与析构 🍷

类的构造与析构分别由类的[构造函数](#)和[析构函数](#)来实现，其中构造函数与析构函数[与类同名](#)；只是[析构函数前多了一个 ~ 符号](#)。

构造函数和析构函数具有普通成员函数的许多共同特性，但还具有一些独特的特性，我们可以把它们归纳成下面几点：

- ①它们都[没有返回值说明](#)，也就是说定义构造函数和析构函数时不需指出类型。
- ②它们不能被继承。 //一代负责一代/每一代不同
- ③和大多数C++函数一样，构造函数可以有缺省参数。
- ④[析构函数可以是虚的\(virtual\)，但构造函数不行。](#)
- ⑤不可取它们的地址。
- ⑥不能用常规调用方法调用构造函数；当使用完全的限定名(带对象名、类名和函数名)时可以调用析构函数。
- ⑦[当定义对象时，编译程序自动调用构造函数；](#)
[当删除对象或到了生命期时，程序自动地调用析构函数。](#)

1、为什么要构造和析构造函数？//比如银行开户和销户，有一系列动作需要做对象的运行与存在需要分配存储空间和初始化，构造函数则是用来为该类的对象实例分配内存空间和初始化数据；而析构与构造相反，它在对象运行完毕时释放对象实例所占用的内存空间。

对象的初始化，比变量的初始化（int i=5）要复杂很多。

所以对象需要构造函数来初始化

2、构造函数与析构函数的运行：

在每次生成类对象（对象实例化）时自动调用构造函数。

在删除对象实例时，及程序执行离开类对象的作用域范围时自动调用析构造函数。

3、构造函数性质

(1) 函数名与类名同

(2) 不能有返回值，包括void

(3) 定义对象时，系统自动调用构造函数

4、构造函数的定义

如果用户不定义构造函数，则系统自动生成类的构造函数和析构函数，只是这时只能分配和释放对象实例的存储空间，而不进行初始化或其他工作。

```
class point {  
    private:  
        int x,y;  
    public:  
        setpoint(int,int);  
};
```

• • •

```
main() {  
    • • •  
}
```

```
func1() {
```

```
»»»» point p1,p2;    //定义类point的对象x1, x2时调用缺省构造函数（系统提供）
```

```
...                //缺省构造函数没有任何参数
```

```
...
```

```
p1=p2;              //什么时候析构
```

```
»»» } //调用系统提供的缺省的析构函数
```


注意: 跟踪调试构造与析构/见视频

```
#include <iostream>
```

```
class point {
```

```
private:
```

```
int x,y;
```

```
public:
```

```
point() {x=9;y=9;} // 一旦自定义, 系统就不提供任何缺省的构造函数
```

```
setpoint(int i, int j) {x=i;y=j;}
```

```
~point() {x=0;y=0;}
```

```
};
```

```
void func1();
```

```
main() { func1(); }
```

```
void func1() {
```

```
point p1,p2; //定义类point的对象x1, x2时调用构造函数
```

```
point p3;
```

```
p1.setpoint(1,1);
```

```
p2.setpoint(2,2);
```

```
p1=p2; //什么时候析构
```

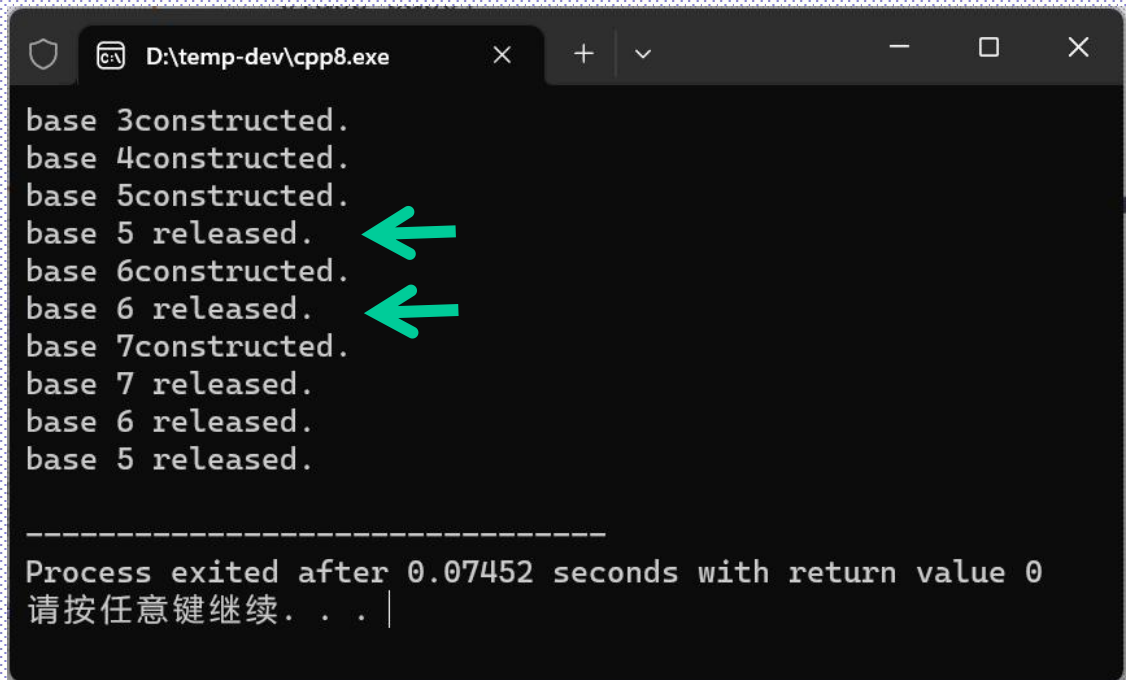
```
}
```

一旦工作了, 就没有了失业金

例子分析:

```
#include <iostream>
using namespace std;
class base{
private:
    int a;
public:
    base(int i) {a=i;cout<<"base "<<a<<"constructed.\n";}
    ~base() {cout<<"base "<<a<<" released.\n";        }
};

int main() {
    base b1(3), b2(4);
    b1=5;
    b2=base(6);
    base b(7);
    return 0;
}
```



```
D:\temp-dev\cpp8.exe
base 3constructed.
base 4constructed.
base 5constructed.
base 5 released.
base 6constructed.
base 6 released.
base 7constructed.
base 7 released.
base 6 released.
base 5 released.

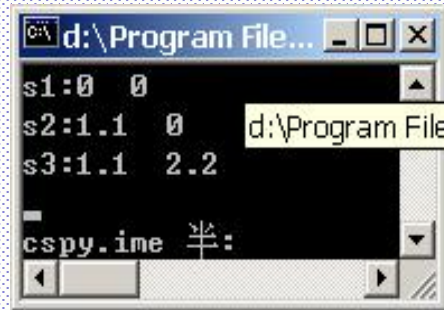
-----
Process exited after 0.07452 seconds with return value 0
请按任意键继续. . .
```

5、带缺省参数的构造函数/视频

```
class complex{  
    private:  
        double real;  
        double imag;  
    public:  
        complex(double r=0.0,double i=0.0);  
        double abscomplex();  
};
```

这时：

```
complex s1;           //全部用缺省值；注意，不是缺省构造函数  
complex s2(1.1);      //只传递一个参数  
complex s3(1.1,2.2);  //传递二个参数
```



s1	object complex	
	real	double 0
s2	imag	double 0
	object complex	
s2	real	double 1.1
	imag	double 0
s3	object complex	
	real	double 1.1
	imag	double 2.2

在上面定义了三个对象s1, s2, s3, 它们都是定义合法的对象, 传递不同的参数, 使它们的私有成员real, imag取得不同的值。

对象s1, 因为在定义时没有传递参数, 所以real, imag均取构造函数的缺省参数值为其赋值, 因此real, imag均为0值。

对象s2, 在定义时只传递了一个参数, 传递的这个参数就给了构造函数中的第一个参量, 而第二个参量取缺省值, 所以对象s2的私有成员real取值为1.1, 而imag为0。

对象s3, 在定义时传递了两个参数, 这两个参数分别给了私有成员real和imag取值为1.1, 2.2。

前面所介绍的缺省参数的用法，不管是单个参数缺省，还是多个参数缺省，都有全缺省的情况，也就是说在函数中所带的参数都是可缺省的。

在许多应用中，经常会遇到这样的情况，在函数所带的参数中，有一部分可以缺省，而有一部分不可缺省。此时所采取的规则是，所有取缺省值的参数必须出现在不取缺省值的参数的右边。也就是说所有的缺省参数必须是参数表中最后的参数。下面的例子非常清楚地描述了这个原则：

- ① `point(int x, int y=0);` `//正确`
- ② `point(int x=0, int y);` `//错误`
- ③ `void f(int i, int j, int k=10);` `//正确`
- ④ `void xyout (char * str, int x=-1, int y=-1);` `//正确`
- ⑤ `void f(int i, int j=10, int k);` `//错误`

某1个定义了缺省，后面都要定义缺省

某1个使用了缺省值，后面都要用缺省值

问题：对于带有参数而不具有默认值的构造函数，如果在定义类对象时没有带参数，会出现怎样情况？（参数列表必须对应）（缺省构造函数）

当自己定义过构造函数后：系统缺省构造函数失效或者说系统不再提供

```
class complex{
    private:
        double real;
        double imag;
    public:
        complex(double, double);
        double abscomplex();
};

complex::complex(double r, double i) {
    real=r;
    imag=i;
}

double abscomplex() {
    return 5;
}

main() {
    complex s1;          complex s2(1.1);          complex s3(1.1, 2.2);
}
```

没有缺省的构造函数了
s1出错

6、析构函数

类的析构函数不接受参数也不返回数值，类只有一个析构函数，不能进行析构函数的重载。

每个类必须有一个析构函数，若没有显式地为一个类定义析构函数，则编译系统会自动地生成一个缺省的析构函数。

对于多数类而言，缺省的析构函数就能满足要求。

如果在某个对象在析构以前要进行一些内部处理，则要显式地定义析构函数。

比如构造时有new的，析构时要delete

7、重载构造函数

类sample有3个构造函数，则这3个构造函数要符合函数重载的要求，不能造成二义性。

```
class sample{  
    public:  
        sample();  
        sample(int, int);  
        sample(double);  
}
```

比较下面3个：

```
sample(double, int);  
sample(double, int);  
sample(int, double);    //可以，不冲突
```


例子:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class sample{
```

```
    public:
```

```
    int i;
```

```
    sample() {i=1;} //如果还需要没参数的，自己写，然后...
```

```
    sample(int j1=0, int j2=0) {i=j1;}
```

```
    sample(double d) {i=int(d);}
```

```
};
```

```
int main(int argc, char* argv[]) {
```

```
    sample s , s1(1,1), s2(1.1);
```

```
}
```

注意：二义性问题讨论：

在定义构造函数时，一定要注意，所定义的多个构造函数之间，在参数的个数或类型上必须存在差异，否则在系统调用时就会出现二义性。

我们在上一节讲了函数的缺省参数，在定义多个构造函数，又使用缺省参数时，特别要注意防止出现二义性。

通过下面的例子，可以看出二义性出现的可能性：

```
class x {  
    //...  
    public :  
        x( );  
        x(int i=0);  
};
```

在这个类的定义中，我们看出：在类中定义了两个构造函数，一个是不带参数的构造函数，一个是带一个整型参数的构造函数。这两个构造函数无论是在所带参数的个数上，还是在所带参数的类型上都不同，是不应该产生二义性的。

下面我们用一个main()函数来使用这个类：

```
main() {  
    //...  
    X one(10); //定义x类的两个对象  
    X two;  
    //...  
}
```

上面定义了两个对象：

①对象one，在定义过程中，传递了一个整型参量10，它与x(int)相匹配，因此在创建对象one的同时，调用构造函数x(int i20)对对象one初始化。

②对象two，在定义过程中，没有传递参数，此时有两种可能的情况：第一种是在创建对象时调用不带参数的构造函数x()来对对象初始化；另一种情况是在创建对象时调用带缺省参数的构造函数x(int I=0)来对对象进行初始化，参数取缺省值0。这两种情况的存在使系统无法确定到底调用哪个构造函数，这就出现了二义性。

8、构造与析构的顺序

```
class CreateAndDestroy {  
public:
```

```
    CreateAndDestroy( int, string );
```

```
    ~CreateAndDestroy();
```

```
private:
```

```
    int data;
```

```
    string memo;
```

```
} zero(0, " zero ");
```

```
CreateAndDestroy :: CreateAndDestroy( int value, string str) {  
    data = value;    memo=str;  
    cout << "Object " << memo << "    constructor"<<endl;  
}
```

```
CreateAndDestroy :: ~CreateAndDestroy() {  
    cout << "Object " << memo << "    destructor " << endl;  
}
```

```
void create( void );                // prototype
```

```
CreateAndDestroy first( 1, " first " ); // global object
```



```

int main() {
    cout << "    (global created before main)" << endl;
    CreateAndDestroy second( 2, " second " );           // local object
    cout << "    (local automatic in main)" << endl;

    static CreateAndDestroy third( 3, " third " ); // local object
    cout << "    (local static in main)" << endl;

    create(); // call function to create objects

    CreateAndDestroy fourth( 4, " fourth " );           // local object
    cout << "    (local automatic in main)" << endl;

}

```



```
// Function to create objects
```

```
void create( void ){
```

```
CreateAndDestroy fifth( 5, " fifth " );
```

```
cout << "    (local automatic in create)" << endl;
```

```
static CreateAndDestroy sixth( 6, " sixth " );
```

```
cout << "    (local static in create)" << endl;
```

```
CreateAndDestroy seventh( 7, " seventh " );
```

```
cout << "    (local automatic in create)" << endl;
```

```
}
```

局部

Second

Fifth

seventh

fourth

静态

third

sixth

全局

Zero

First


```
C:\WINDOWS\system32\cmd.exe
E:\temp>project1
Object zero    constructor
Object first   constructor
(global created before main)
Object second  constructor
(local automatic in main)
Object third   constructor
(local static in main)
Object fifth   constructor
(local automatic in create)
Object sixth   constructor
(local static in create)
Object seventh constructor
(local automatic in create)
Object seventh destructor
Object fifth   destructor
Object fourth  constructor
(local automatic in main)
Object fourth  destructor
Object second  destructor
Object sixth   destructor
Object third   destructor
Object first   destructor
Object zero    destructor
```

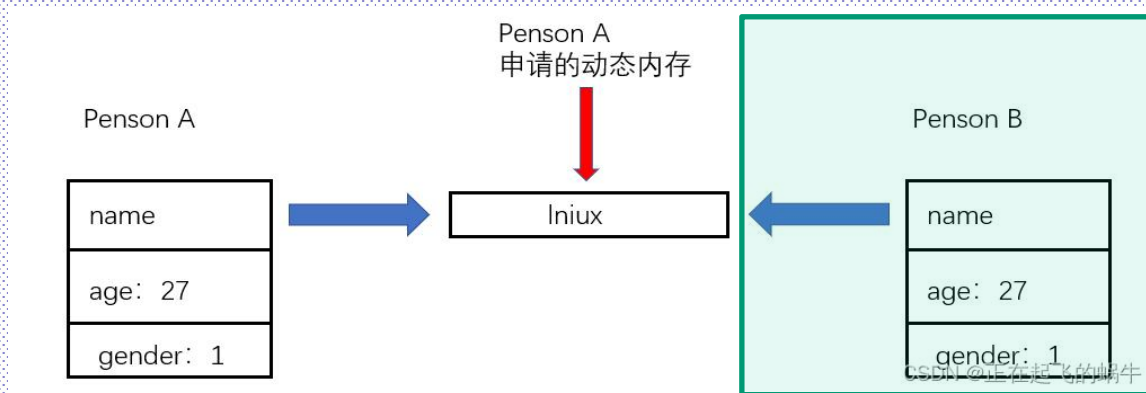
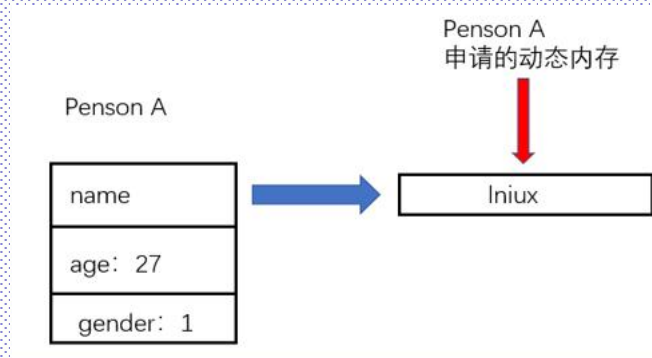

9-0、浅拷贝：

简单的讲，对象拷贝时，对指针变量做处理的是深拷贝；
不对指针变量做处理的是浅拷贝；

```
class Person{  
public:  
    string *name;           //姓名，是一个指针变量  
    int age;  
    int gender;             //性别：0表示男，1表示女  
};
```

```
Person A;  
A.name = new string("linux");  
A.age=27;  
A.gender = 1;
```

```
Person B;  
B = A;  
delete A.name;
```



9-0、浅拷贝：

简单的讲，对象拷贝时，对指针变量做处理的是深拷贝；
不对指针变量做处理的是浅拷贝；

```
class Person{
public:
    string *name;    //姓名，是一个指针变量
    int age;
    int gender;      //性别：0表示男，1表示女
};
```

Person A;

A.name = new string("linux");

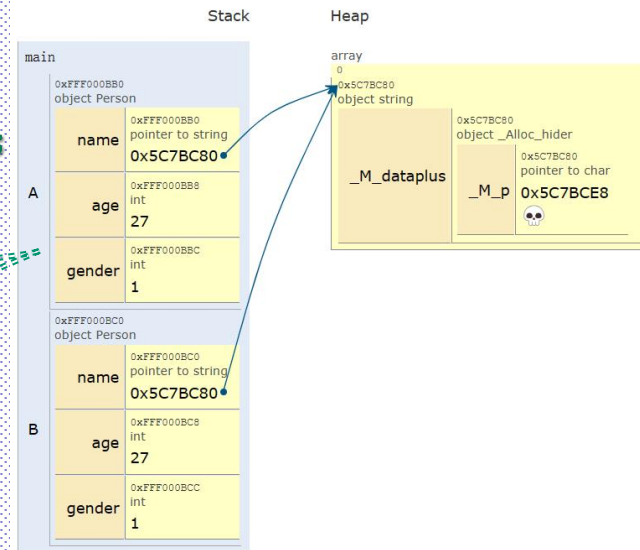
A.age=27;

A.gender = 1;

Person B;

B = A;

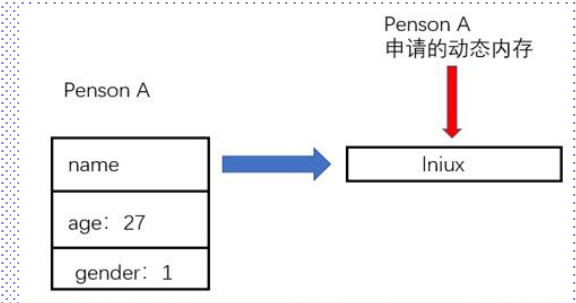
delete A.name;



9-0、深拷贝：

简单的讲，对象拷贝时，对指针变量做处理的是深拷贝；
不对指针变量做处理的是浅拷贝；

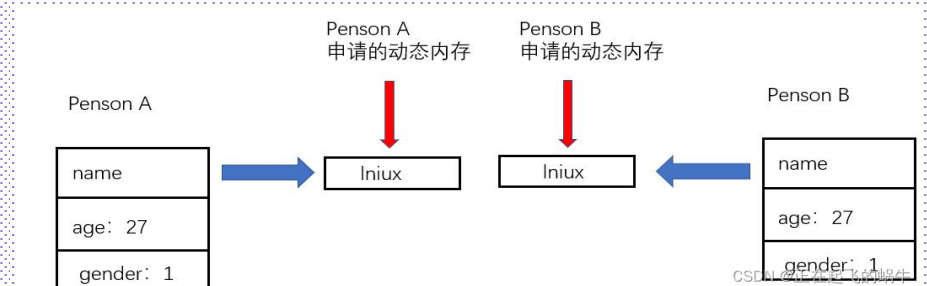
```
class Person{  
public:  
    string *name;        //姓名，是一个指针变量  
    int age;  
    int gender;          //性别：0表示男，1表示女  
};
```



```
Person A;  
A.name = new string("linux");  
A.age=27;    A.gender = 1;
```

```
Person B;  
B.name = new string(*(A.name));  
B.age=A.age;    B.gender=A.gender;
```

```
delete A.name;
```



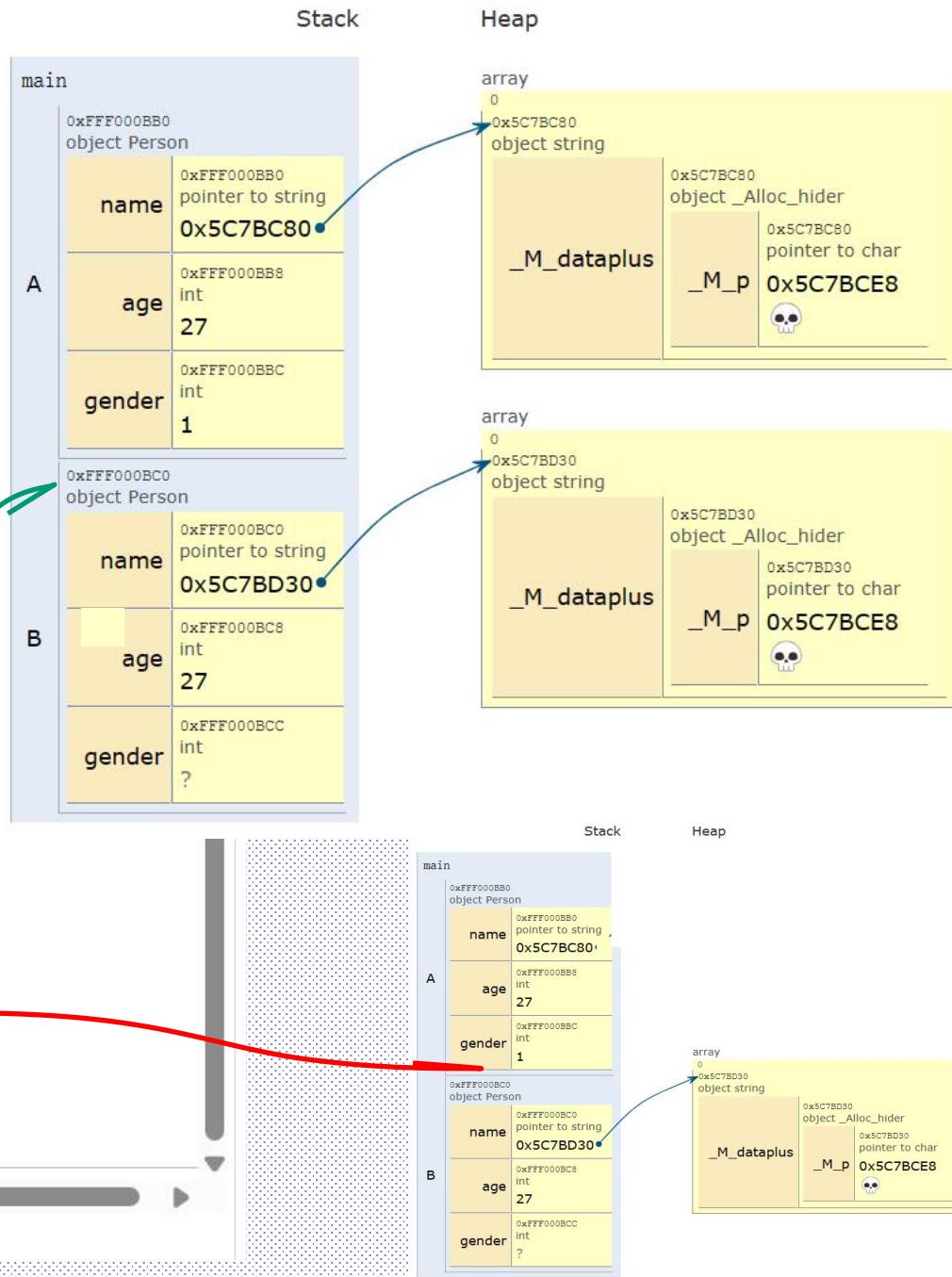
9-0、深拷贝：

简单的讲，对象拷贝时，对指针变量
不对指针变量做处理的是浅拷贝；

C++ (C++20 + GNU extensions)
[known limitations](#)

```
11     Person() { }  
12     //Person() {string* n, int a, int  
13 };  
14  
15  
16 int main() {  
17     Person A;  
18     A.name = new string("linux");  
19     A.age=27;  
20     A.gender = 1;  
21  
22     Person B;  
23     B.name = new string(*(A.name));  
24     B.age=A.age;  
25     B.gender=A.gender;  
26  
27     delete A.name;  
28  
29     return 0;  
30 }
```

[Edit this code](#)

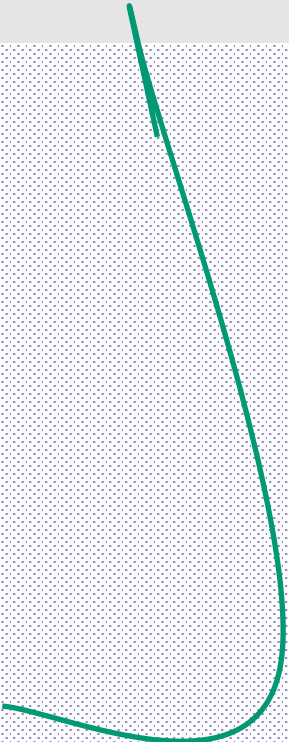


9-0、深拷贝：

深拷贝在构造函数里

```
Student::Student(char *name1,char *stu_no1,int score1){  
    name=new char[strlen(name1)+1];stu_no=new char[strlen(stu_no1)+1];  
    strcpy(name,name1);strcpy(stu_no,stu_no1);  
    score=score1;}
```

```
class Student{  
    private:  
        int stu score;  
        char *stu_no;  
        char *name;  
    public:  
        Student();  
        Student(char*,char*,int);  
        ~Student();  
};
```



9、拷贝构造函数：双要素[复制+新对象]

拷贝构造函数是一种**特殊的构造函数**，用来**将一个已存在对象的值赋给一个新对象**（当新对象建立时）。 //用对象复制新对象（克隆）

它的特殊功能是将**参数对象的值逐域拷贝到新创建的对象中**。

(1) **缺省的拷贝构造函数**

同其他构造函数一样，如果没有用户自定义的拷贝构造函数，**系统将提供一个缺省的拷贝构造函数**。

拷贝构造函数的定义与使用情况：

(1) 定义时：

```
point p2(p1);    //显式调用  //int i(3);
```

```
point p3=p1;    //复制式调用；注意这里不是=赋值；因为是新建对象
```

(2) 参数传递时：

当对象作为参数传递时，在实参和形参的传递过程中，拷贝构造函数将被调用

(3) **什么时候还会有？返回值**

例子:

```
class point{
    int px,py;
public:
    point(int a,int b) {px=a;py=b;}
    point(const point &tp) {
        px=tp.px;py=tp.py;
        cout<<"copy constructor"<<endl;
    }
    void print() {cout<<px<<" "<<py<<endl;}
    void setpx(int i) {px=i;}
    int getpx() {return px;}
};
```

```
int main(int argc, char* argv[]) {
    point p1(30,40);
    point p2(p1);
    point p3=p1;
    p3.setx(9);
    p1.print();
    p2.print();
    p3.print();
}
```


例子：//返回值为Point类对象的函数

```
point fun2() {  
    Point ta(1, 2);  
    return ta;  
}
```

```
void fun1(point x) {  
    cout<<"fun1"<<endl;  
}
```

```
class point{  
    int px,py;  
public:  
    point(int a,int b) {px=a;py=b;}  
    point(const point &tp) {  
        px=tp.px;py=tp.py;  
        cout<<"copy constructor"<<endl;  
    }  
    void print() {cout<<px<<" "<<py<<endl;}  
    void setpx(int i) {px=i;}  
    int getpx() {return px;}  
};
```

```
int main() {  
    point p1(4, 5);
```

```
    cout<<"=====0======"<<endl;
```

① point p2=p1; //情况一，用A初始化B。第一次调用复制构造函数

```
    cout<<"=====1======"<<endl;
```

fun1(p2); //情况二，对象B作为fun1的实参。第二次调用复制构造函数

```
    cout<<"=====2======"<<endl;
```

fun2(); //情况三，函数的返回值是类对象，返回return时调用复制构造函数

```
    cout<<"=====3======"<<endl;
```

② p2 = fun2(); //情况三; =只是赋值

```
    cout<<"=====4======"<<endl;
```

```
    cout << p2.getpx() << endl;
```

```
    return 0;
```

```
}
```

①的等号和②的等号

```
=====0=====
copy constructor
=====1=====
copy constructor
fun1
=====2=====
copy constructor
=====3=====
copy constructor
=====4=====
1
```

(2) 用户自定义拷贝构造函数

自定义拷贝构造函数的一般形式: `point(const point &op) { }`

可以在类定义中定义一个拷贝构造函数, 其格式为: `X::X(const X&)`, 若X为类名, 则拷贝构造函数带一个类型为X&的参数。

例子:

```
class point{
    int x,y;
    public:
        point(int a,int b) {x=a;y=b;}
        point(const point &op){
            x=op.x+1;
            y=op.y+1;
            cout<<"be executed"<<endl;
        }
        void print() {cout<<x<<" "<<y<<endl;}
        void setx(int i) {x=i;}
};
```

//为什么要用const??

//point(point op)系统不允许

//为什么引用: 1、快;

//2、拷贝构造自身还未定义

引用的对象, 这样, 在参数传递过程中就不需要执行拷贝初始化构造函数, 这将会改善程序的运行效率。(节省空间和时间)

```
int main(int argc, char* argv[]) {  
    point p1(30,40);  
    point p2(p1);  
    point p3=p1;  
    p3.setx(9);  
    p1.print();  
    p2.print();  
    p3.print();  
    return 0;  
}
```

9、拷贝构造函数：深拷贝

```
class Student{  
public:  
    //构造函数  
    Student(char* p);  
    //拷贝构造函数  
    Student(const Student& stu);  
  
    char *name;  
};
```

//调用方法1

Object obj1(...);

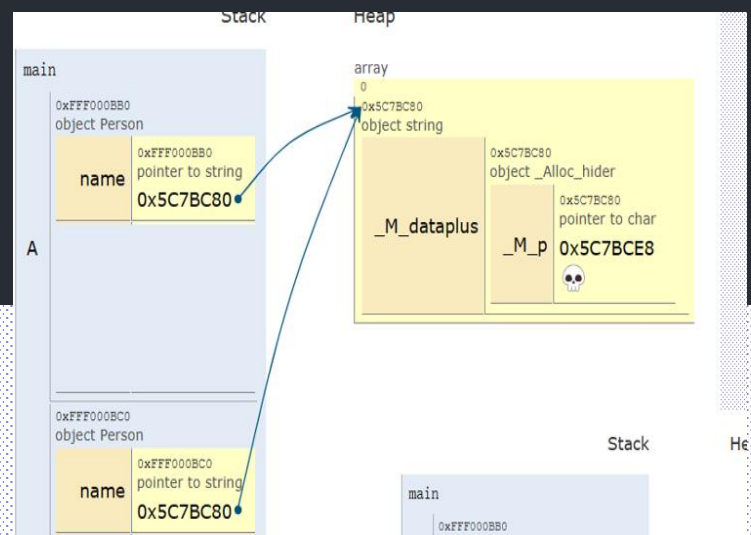
Object obj2(obj1);

或者写成: Object obj2 = obj1;

//缺省的构造函数和拷贝构造函数:

//obj1和obj2都指向同一个name的内容空间

```
Student(const char* myName)  
{  
    int len = strlen(myName);  
    name = new char[len + 1]{0};  
    strcpy_s(this->name, len+1, myName);  
  
    cout << "构造: " << hex << (int)name << endl;  
}  
  
Student(const Student& stu)  
{  
    int len = strlen(stu.name);  
    name = new char[len + 1];  
    strcpy_s(name, len + 1, stu.name);  
    cout << "调用拷贝构造函数" << hex << (int)name << endl;  
}  
  
~Student()  
{  
    if (name)  
    {  
        cout << "析构: " << hex << (int)name << endl;  
        delete[] name;  
        name = NULL;  
    }  
}
```



四、对象数组与对象指针

1、对象数组

例子：

```
class exam{
    int x;
public:
    void set_x(int n) {x=n;}
    int get_x() {return x;}
};

main(int argc, char* argv[]) {
    exam ob[4];    //调用几次构造函数?
    //exam *pob[4]; //调用几次构造函数?
    int j;
    for (j=0;j<4;j++) cout<<ob[j].get_x()<<' ';
    cout<<endl;
}
```

例子：带有自定义构造函数

```
#include <iostream>
```

```
class exam{
```

```
    int x;
```

```
public:
```

```
    exam(int n) { x=n;}
```

```
    void set_x(int n) {x=n;}
```

```
    int get_x() {return x;}
```

```
};
```

```
int main(int argc, char* argv[]) {
```

```
    exam ob[4]={11, 22, 33, 44};    //初始化数据，构造函数是哪一个？调用几次？
```

```
    //exam bb(1, 2);                //找不到对应的构造函数
```

```
    //exam oo;
```

```
    int j;
```

```
    for (j=0; j<4; j++) cout<<ob[j].get_x()<<' ';
```

```
    cout<<endl;
```

```
}
```

例子三:

```
class exam{
    int x,y;
public:
    exam() {x=5;y=5;}
    exam(int n,int m) {x=n;y=m;}
    void set_x(int n) {x=n;}
    int get_x() {return x;}
    int get_y() {return y;}
};
```

```
main(int argc, char* argv[]) {
    exam ob0(4,4);
    exam ob1[5];
    //exam ob1[5]={exam(4,4), exam(3,3), exam(2,2), exam(1,1), exam(0,0)};
    cout<<ob0.get_x()<<' ' <<ob0.get_y()<<endl;
    int j;
    for (j=0;j<5;j++) cout<<ob1[j].get_x()<<' ' <<ob1[j].get_y()<<endl;
}
```


2、对象指针

如上例中: exam ob0, **obp*;

注意: 如何使用obp? 对象指针与构造函数怎样关系?

```
#include <iostream>
class exam{
public:
    int x,y;
    exam() {x=5;y=5;}
    exam(int n,int m) {x=n;y=m;}
    void set_x(int n) {x=n;}
    int get_x() {return x;}
    int get_y() {return y;}
};
```

2、对象指针

如上例中: exam ob0, *obp;

注意: 如何使用obp? 对象指针与构造函数怎样关系?

```
#include <iostream.h>
class exam{
public:
    int x,y;
    exam() {x=5;y=5;}
    exam(int n,int m) {x=n;y=m;}
    void set_x(int n) {x=n;}
    int get_x() {return x;}
    int get_y() {return y;}
};
```

```
int main(int argc, char* argv[]) {
    exam ob0(4,4);
    exam ob1[5];
    exam *ob2; //有没有调用构造函数?
    cout<<ob0.get_x()<<endl;
    for (int j=0;j<5;j++)
        cout<<ob1[j].get_x()<<<<endl;
    cout<<ob2->x<<endl; //有问题吗?
    ob2->x=99;
    ob2->y=98;
    cout<<ob2->x<<" "<<ob2->y<<endl;
}
```

需要: ob2=new exam();
delete ob2;

3、this指针: **this**指针是指向 **当前对象** 的指针; this 就是我的指针

```
#include <iostream>
class exam{
    int i;
public:
    void load(int val) {this->i=val;} //等同于i=val;
    int get() {return this->i;}
};
main(int argc, char* argv[]) {
    exam ob1, ob2;
    ob1.load(100);
    cout<<ob1.get()<<endl;
    ob2.load(99);
    cout<<ob2.get()<<endl;
}
```



一种情况就是, 在类的非静态成员函数中①返回类对象本身的时候, 直接使用

return *this;

```
void load(int i) {this->i=i;}
```

另外一种情况是②当参数与成员变量名相同时, 如this->i = i (不能写成 i = i)。

string类

```
#include<string>
```

```
string str1, str2;  
string str3( "china" );  
string str4= " china" ;           //
```

运算符已经经过重载，支持+、=、==、>、<等大小比较
支持数组[]，支持输入输出<<, >>

```
str1=str2  
str1+str2  
str1==str2  
cin>>str1  
cout<<str2
```

string 读取含空格的字符串：

```
string s;  
getline(cin, s); 带空格
```

运算符重载：operator+()

一个运算符对应一个函数，函数可以重载

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string strinfo="Please input your name:";
    cout<<strinfo ;      cin>>strinfo;
    if( strinfo == "winter" )      cout << "you are winter!"<<endl;
    else if( strinfo != "wende" )  cout << "you are not wende!"<<endl;
    else if( strinfo < "winter")   cout << "your name should be ahead of
winter"<<endl;
    else cout << "your name should be after of winter"<<endl;

    strinfo += " , Welcome to China!";
    cout << strinfo<<endl;

    string strtmp = "How are you? " + strinfo;
    cout<<strtmp<<endl;
    return 0;
}
```

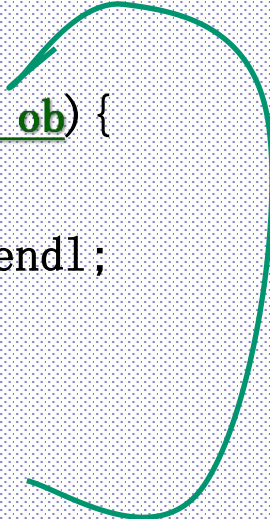
五、函数中对象参数的传递

1、传值=对象(或普通变量)为参数

```
class exam{  
    int i;  
public:  
    void load(int val) {i=val;}  
    int get() {return this->i;}  
};
```

```
void printval(exam ob) {  
    cout<<"val:";  
    cout<<ob.get()<<endl;  
}
```

//调用拷贝构造函数



```
main() {  
    exam ob1, ob2;  
    ob1.load(100);  
    printval(ob1);  
    ob2.load(99);  
    printval(ob2);  
  
}
```

2、传地址=对象指针(或普通变量指针)为参数

问题：是否会影响参数对象本身？

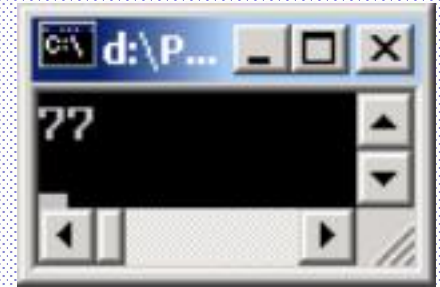
```
class exam{
    int i;
public:
    void load(int val) {
        i=val;}
    int get() {
        return this->i;}
};
```

```
void setval(exam *ob) {
    ob->load(77);
} // ob指向的是main里的ob1
```

```
void setvalarr(exam *ob) {
    ob[1].load(88);
} // ob指向的是main里的数组首地址
```

```
main() {
    exam ob1, ob2;
    ob1.load(100);
    setval(&ob1);
    cout<<ob1.get()<<endl;

    exam ob3[5];
    setvalarr(ob3);
}
```



3、传引用=对象引用为参数

问题：是否会影响参数对象本身？

```
class exam{
    int i;
public:
    void load(int val) {
        i=val;}
    int get() {
        return this->i;}
};
```

```
void setval(exam &ob) {
    ob.load(77);
}
```

//ob就是main里的ob1

```
int main() {
    exam ob1, ob2;
    ob1.load(100);
    setval(ob1);
    cout<<ob1.get()<<endl;
    return 0;
}
```



七、常类型 🐼

1. 一般常量

一般常量是指简单类型的常量。这种常量在定义时，修饰符const可以用在类型说明符前，也可以用在类型说明符后。如：

```
int const x=2; 或      const int x=2;
```

```
int const x=2;
main() {
    const int y=3;
    cout<<x<<" "<<y<<" ";
}
```

定义或说明一个常数组可采用如下格式：

＜类型说明符＞ const ＜数组名＞[＜大小＞]…

或者

const ＜类型说明符＞ ＜数组名＞[＜大小＞]…

例如：

```
int const a[5]={1, 2, 3, 4, 5};
```

2. 常指针[同前面新特性]

使用const修饰指针时，由于const的位置不同，而含意不同。下面举两个例子，说明它们的区别。

下面定义的一个指向字符串的常量指针：`char * const prt1 = stringprt1;`
其中，prt1是一个常量指针。因此，下面赋值是非法的。

```
prt1 = stringprt2;
```

而下面的赋值是合法的：

```
*prt1 = "m";
```

因为指针prt1所指向的变量是可以更新的，不可更新的是常量指针prt1所指的方向(别的字符串)。

下面定义了一个指向字符串常量的指针：`const char * ptr2 = stringprt1;`

其中，ptr2是一个指向字符串常量的指针。ptr2所指向的字符串不能更新的，而ptr2是可以更新的。因此，

```
*ptr2 = "x";
```

是非法的，而：

```
ptr2 = stringprt2;
```

是合法的。

所以，在使用const修饰指针时，应该注意const的位置。定义一个指向字符串的指针常量和定义一个指向字符串常量的指针时，const修饰符的位置不同，前者const放在*和指针名之间，后者const放在类型说明符前。

3. 常引用

使用const修饰符也可以说明引用，被说明的引用为常引用，该引用所引用的对象不能被更新。其定义格式如下：

`const <类型说明符> & <引用名>`

例如：

`const double & cvar;`

在实际应用中，常指针和常引用往往用来作函数的形参，这样的参数称为常参数。

在C++面向对象的程序设计中，指针和引用使用得较多，其中使用const修饰的常指针和常引用用得更多。使用常参数则表明该函数不会更新某个参数所指向或所引用的对象，这样，在参数传递过程中就不需要执行拷贝初始化构造函数，这将会改善程序的运行效率。（节省空间和时间）

下面举一例子说明常指针作函数参数的作法。

```
#include  
const int N = 6;  
void print(const int *p, int n);
```

```
main() {  
    int array[N];  
    for (int i=0; i<N;i++) cin>>array[i];  
    print(array, N);  
}
```

```
void print(const int *p, int n) {  
    cout<<"{"<<*p;  
    for (int i=1; i<N;i++) cout<<", "<<*(p+i);  
    cout<<"}"<  
}
```

4、常对象

常对象是指对象常量，定义格式如下：

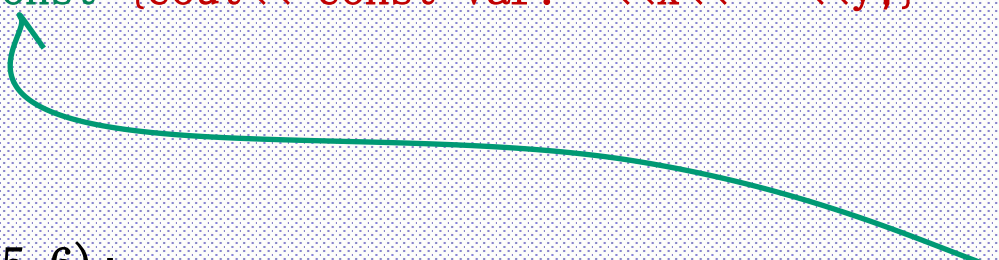
```
class A;  
const A a;  
A const a;
```

定义常对象时，同样要进行初始化，并且该对象不能再被更新，修饰符const可以放在类名后面，也可以放在类名前面。

例子:

```
class sample{
public:
    int x,y;
    sample(int x=0,int y=0) {
        sample::x=x;           //如果没有左边的sample:: ?
        sample::y=y;           //this->x=x
    }
    void disp() {cout<<x<<" "<<y;}
    //void disp() const {cout<<"const var: "<<x<<" "<<y;}
};

main() {
    const sample s(5,6);
    s.disp();                  //绿色部分会有问题, why? //需要常成员函数
    s.x=55;
}
```



5、常对象成员和常数据成员

类型修饰符可以说明数据成员。

由于**const**类型对象必须被初始化，并且不能更新，因此，在类中说明了const数据成员时，**只能通过成员初始化列表的方式**来生成构造函数对数据成员初始化。

下面通过一个例子讲述使用成员初始化列表来生成构造函数。

```
class A{
public:
    A(int i);
    void print();
    const int &r;
private:
    const int a;
    static const int b;    //1作用域:类内 2:所有对象公/共用一个static空间
};
```

`const int A::b=10;` //为什么如此定义，在静态成员中详解

```
A::A(int i):a(i), r(a){ }
void A::print() {cout<<a<<endl;}
```

```
main() {
    A a1(100), a2(0);
    a1.print();
    a2.print();
}
```

```
f(int a) {
    auto int b=0;
    static int c=3; //静态变量，函数调用结束，不会消
    b=b+1; c=c+1;
    return(a + b + c);
}
main( ) {
    int a=2, i;
    for(i=0; i<3; i++) printf( "%d " , f(a));
    return 0;
}
```

在该程序中，说明了如下三个常类型数据成员：

```
const int & r;  
const int a;  
static const int b;
```

其中，r是常int型引用，a是常int型变量，b是静态常int型变量。程序中对静态数据成员b进行初始化。

值得注意的是构造函数的格式如下所示：

```
A(int i):a(i),r(a) { }
```

其中，冒号后边是一个数据成员初始化列表，它包含两个初始化项，用逗号进行了分隔，因为数据成员a和r都是常类型的，需要采用初始化格式。

通常情况下，

```
A(int i):a(i),r(a) { }
```

等价于：

```
A(int i): {a=i,r=a;}
```

但在这里不行。因为后一种的写法表示了a和r可变。

6、常成员函数

使用const关键字进行说明的成员函数，称为常成员函数。

只有常成员函数才有资格操作常量或常对象；

没有使用const关键字说明的成员函数不能用来操作常对象。

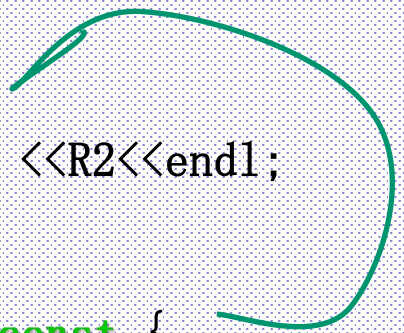
常成员函数说明格式如下：

<类型说明符> <函数名> (<参数表>) const;

其中，const是加在函数说明后面的类型修饰符，它是函数类型的一个组成部分，因此，在函数实现部分也要带const关键字。下面举一例子说明常成员函数的特征。

```
class R{
public:
    R(int r1, int r2) { R1=r1; R2=r2; }
    void print();
    void print() const;
private:
    int R1, R2;
};
```

```
void R::print() {  
    cout<<R1<< “,” <<R2<<endl;  
}  
  
void R::print() const {  
    cout<<“const:”<<R1<< “,” << R2<<endl;    //R1=5;  
}  
  
int main() {  
    R a(5, 4);  
    a.print();  
    const R b(20, 52);  
    b.print();    return 0;  
}
```



该例子的输出结果为：

5, 4

const:20;52

该程序的类声明了两个成员函数，其类型是不同的(其实就是**重载成员函数**)。有带const修饰符的成员函数处理const常量，这也**体现出函数重载**的特点。

说明:

- 1) 如果将一个对象说明为常对象, 则通过该对象只能调用它的常成员函数, 而不能调用普通的成员函数。常成员函数是常对象唯一的对外接口。
- 2) 常成员函数不能更新对象的数据成员, 也不能调用该类中的普通成员函数。
- 3) 普通对象能调用常成员函数。 (如果对应的普通成员函数没有)

以上红色部分在不同的编译器中可能为警告或错误

```
class R{
public:
    R(int r1, int r2) { R1=r1; R2=r2; }
    void print() const;
private:
    int R1, R2;
};
void R::print() const {
    cout<<"const:"<<R1<<"", "<< R2<<endl; //R1=5;
}
int main(){
    R a(5, 4);    a.print();
    const R b(20, 52);    b.print();
    return 0;
}
```

```
const:5, 4
const:20, 52
Press any key to continue_
```

六、静态成员

在类定义中，它的成员(包括数据成员和函数成员)可以声明成静态的，即用关键字`static`，这些成员就被称为静态成员。

它的特征是不管这个类创建了多少个对象，其静态成员只有一个副本，这个副本被(这个类的)所有的对象共享。

静态成员在类中有两种情况，即静态数据成员和静态函数成员。

1、静态的数据成员：

在一个类中，若将一个数据成员说明成`static`，这种成员称为静态数据成员。无论建立多少个该类的对象，都只有一个静态数据的拷贝。当这个类的第一个对象被建立时，所有`static`数据都被初始化，并且，以后再建立对象时，就不需再对其做初始化。

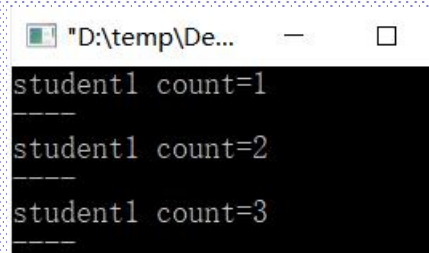
注意：静态成员的初始化方法与位置：应在该类的任何对象建立之前就存在。
比如一个班级的教室，一个班级的班会费银行卡：**必须先所有对象之前存在**

所有对象使用同一个静态成员空间。(所有对象的静态成员只有一个拷贝)


```

class student{
    static int count;
    int studentno;
public:
    student() {
        count++;
        studentno=count;
    }
    void print() {
        cout<<"student"<<studentno<<" ";
        cout<<"count="<<count<<endl;
    }
};

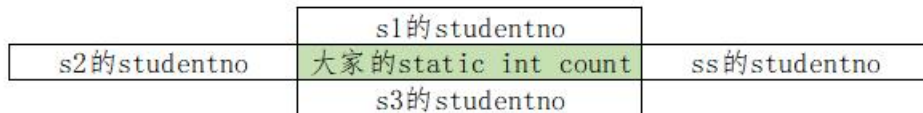
```



```

"D:\temp\De...
-----
student1 count=1
-----
student1 count=2
-----
student1 count=3
-----

```



int student::count=0; //私有可以?

```

int main() {
    student s1;
    s1.print();
    cout<<"----"<<endl;

```

```

//student *ss;
//ss=new student()
//ss->print();
//delete ss;

```



```

选定 d:\Progr...
-----
student1 count=1
-----
student2 count=2
student1 count=3
-----
student1 count=4
-----

```

```

student s2;
s1.print();
cout<<"----"<<endl;
student s3;
s1.print();
cout<<"----"<<endl;
return 0;

```

```

}

```


注意：

(1) 静态数据成员属于类，**而不是属于某一个对象实例**，因此可以用：

“**类名::静态成员**”来访问。如上例中可以：

```
cout<<student::count<<endl;
```

(2) 静态成员不能在类中进行初始化，是在创建时初始化的。

(3) **静态成员应该在该类的任何对象被建立之前就存在。**

(4) 使用静态成员可以使程序员**不必使用全局变量**，有利于面向对象的封装。

注意：全局变量与类静态成员的区别

2、静态的成员函数：主要用于对静态数据成员或全局变量的访问

```
class student{
    static int count;
    int studentno;
public:
    student() {
        count++;studentno=count;
    }
    static void init() {count=1;}
    static void print() {
        //cout<< "student" <<studentno<< " " ;
        //能否访问非静态成员(不能)
        //cout<<"student"<<this->studentno<<" ";
        //静态成员函数中有this指针？(无) （都是因为静态不属于某个具体对象）
        cout<<"count="<<count<<endl;
    }
};
```

```
int student::count=0;
```

```
main() {  
    //student::init();    //如有这一句，结果有何不同？， 初始化为1  
    student s1;  
    s1.print();  
    cout<<"-----"<<endl;  
    student s2;  
    s1.print();  
    cout<<"-----"<<endl;  
    student s3;  
    s1.print();  
    cout<<"-----"<<endl;  
    student::print();  
}
```

注意:

- (1) 静态成员函数可以在类内定义，也可以在类外定义，在类外定义时不用再写static前缀。（类似缺省参数，声明时起作用）
- (2) 静态成员函数可以在建立任何对象之前处理静态数据成员，这是普通成员函数不能实现的。如：`student::init()`;
- (3) 以下两个不会产生冲突，因为前者是类的静态成员函数（注意前没有static, 1中要求），后者是类外的普通函数。

```
void student::init() {  
    count=1;  
}  
  
static void init() {  
    student::count=1;  
}
```

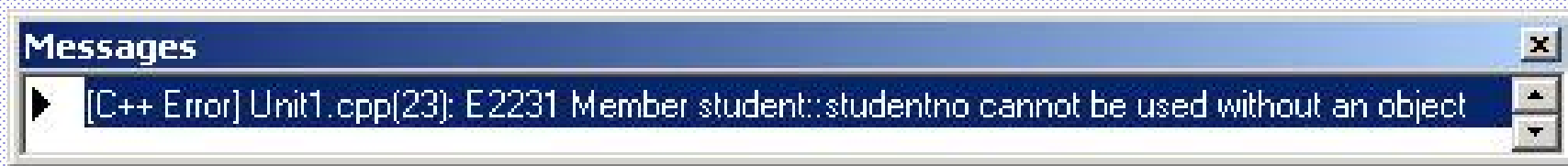
- (4) 静态成员函数没有this指针，因为它不与特定的对象联系。（如班会费银行卡）
- (5) 一般而言，静态成员函数不（直接）访问类中的非静态成员（因为不代表某个对象实例）。

若需要，必须通过参数传递对象名或对象指针访问。

```
static void print (student &w) {  
    w.studentno=3;  
}
```

例子：在静态成员函数中直接使用非静态成员

```
void student::init() {count=1;studentno=5;}
```



正确方法：

```
void student::init(student &w) {count=1; W.studentno=5;}
```

七、友元

1、私有、公有、和友元

私有：只能在类的定义范围内，即该类的成员函数

公有：

友元：可以访问类对象私有成员的非当前类成员函数

2、友元函数(外部函数)

友元函数不是当前类的成员函数

友元函数独立于当前类，是外部函数；或者是其他类的成员函数

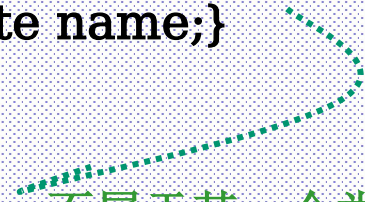
友元会增加数据访问的灵活性

友元会使数据封装性受到削弱，导致程序安全性变差

```
class Box{  
    double width;  
    double length;  
public:  
    void setWidth( double wid );  
    friend void printWidth( Box box ); //print() 不是类的成员函数；不属于类  
};
```

一个例子: 外部函数作为友元

```
class girl{
    char *name;  int age;
public:
    girl(char *n,int d){
        name=new char[strlen(n)+1];
        strcpy(name,n);  age=d;
    }
    friend void disp(girl &);  //if no keyword friend?
    ~girl() {delete name;}
};
```



//disp是外部函数; 不属于某一个类

```
void disp(girl &x){
    cout<<"girl\'s name is:"<<x.name<<" ,age"<<x.age<<endl;
}
```

```
int main(){
    girl e("Chen XingWei",18);
    disp(e);  return 0;
}
```


注意:

(1)、一个类的友元函数不是该类的成员函数，所以在外部定义该函数时，不用::符号，尽管类定义中有友元函数的原型，但友元函数仍然不是成员函数。

(2)、由于友元函数不是该类的成员函数，所以不能直接访问成员，如上例中：

`name="XXXXXXX";`

也不能使用this指针，如：

`this.name="XXXXXXXXXXXXXX";`

而应该通过入口参数来操作，如上例中参数为x, 则有：

`x.name="XXXXXXXXXX";`

```
//disp是外部函数
void disp(girl &x){
    cout<<"girl\'s name is:"<<x.name<<",a
}
```

(3)、当一个函数需要访问多个类时，就可以定义友元函数

(4)、要让B成为A的友元，A要在自己的定义中显式申明B为友元。

友元关系不对称也不能传递。

A是B的友元，B是不是A的友元？

A是B的友元，B是C的友元，A是不是C的友元？

(5)、讨论：友元关系会不会破坏信息隐藏和降低面向对象设计方法的价值

3、友元成员（另一个类的成员函数作为友元）

上面所举的例子中的友元函数对两个类来说都是外部函数。

除了一般的函数可以作为某个类的友元外，一个类的成员函数也可以作为另一个类的友元。

`class girl;` //如果没有这一句

```
class boy{
    char *name;  int age;
public:
    boy(char *n,int d){
        name=new char[strlen(n)+1];
        strcpy(name,n);  age=d;
    }
    void disp(girl &);
    ~boy() {delete []name;}
};
```

```
class girl{
    char *name;
    int age;
public:
    girl(char *n,int d){
        name=new char[strlen(n)+1];
        strcpy(name,n);  age=d;
    }
    friend void boy::disp(girl &);
    ~girl() {delete []name;}
};
```

```
void boy::disp(girl &g1){  
    cout<<"boy\'s name is:"<<name<<",age"<<age<<endl;  
    cout<<"girl\'s name is:"<<g1.name<<",age"<<g1.age<<endl;  
}
```

```
main(){  
    girl g("Chen XingWei",18);  
    boy b("qian Wei",17);  
    b.disp(g);  
    return 0;  
}
```

4、友元类（整个类作为友元）

以上的例子是将函数作为友元。

同样可以将一个类作为另一个类的友元，当一个类被说明为另一个类的友元时，它的所有的成员函数都成为另一个类的友元函数。友元类定义：

```
class y{ };  
class x{ friend y; }
```

```
class girl; //如果没有这一句  
class boy{  
    char *name; int age;  
public:  
    boy(char *n,int d){  
        name=new char[strlen(n)+1];  
        strcpy(name,n); age=d;  
    }  
    void disp(girl &);  
    void print(girl &);  
    ~boy() {delete []name;}  
};
```

```
class girl{  
    char *name;  
    int age;  
    friend boy;  
public:  
    girl(char *n,int d){  
        name=new char[strlen(n)+1];  
        strcpy(name,n); age=d;  
    }  
    ~girl() {delete []name;}  
    //friend void boy::disp(girl &);  
    //friend void boy::print(girl &);  
};
```

```
void boy::disp(girl &g1) {  
    cout<<"boy\'s name is:"<<name<<",age"<<age<<endl;  
    cout<<"girl\'s name is:"<<g1.name<<",age"<<g1.age<<endl;  
}
```

```
main() {  
    girl g("Chen XingWei",18);  
    boy b("qian Wei",17);  
    b.disp(g); return 0;  
}
```

八、类的组合：类成员对象

例子一：一种构造的参数列表

```
class c_temp{
    public:
        int i, j;
        c_temp(int i1, int j1):i(i1), j(j1) { }
        //c_temp(int i1, int j1) {i=i1; j=j1;} //等价
};
```

```
main() {
    c_temp c1(3, 4);
    cout<<c1.i<<" , "<<c1.j<<endl;

}
```

可以[扩展为一个类对象成员的初始化](#):

在初始化列表中完成, 因为:

类的对象成员是类对象的一部分, 要先于类对象之前完成

```
class Computer{  
    类1 类1对象s1;    //先有机箱  
    类2 类2对象s2;    //先有主板  
    类3 类3对象s3;    //先有CPU  
}x1;
```

初始化列表s1...sn的参数包含在(参数0)里

Computer::Computer(参数表0): 成员s1(参数表1), ..., 成员sn(参数表n)

```
{  
2 ...    //开始组装电脑  
}
```

1

也就是说, 要有x1对象, 先要有子/成员对象x1.s1 x1.s2 x1.s3

1 采购部件 (运行子对象构造函数) -> 2 然后才能组装成品 (构造自身: 运行自己构造函数)

一个例子:

```
#include <iostream>
```

```
class Person{  
    string name;  
    string phoneno;  
    string PID;  
public:  
    Person(char *s) {  
        ...  
    }  
    void print() {  
        ...  
    }  
};
```

```
class Currency;
```

```
class Date;
```

```
class SavingAccount{
```

```
public:
```

```
SavingAccount(参数表0): saver(...), balance(...), openday(...) { 然后才是这里 }
```

```
~SavingAccount();
```

```
void print();
```

```
private:
```

```
Person saver;    //成员1: 其他类Person的对象
```

```
Currency balance; //成员2: 其他类Currency的对象
```

```
Date openday;    //成员3: 其他类Date的对象
```

```
}
```

```
main() {
```

```
    SavingAccount sal(...);
```

```
    sal.disp(); return 0;
```

```
}
```

//先构造3个子对象，后构造对象自身sal

//析构相反顺序

1

先调用对应的三个构造函数
生成3个对象成员

2

注意：初始化列表里为成员变量名；不是类名

初始化列表的参数包含在(参数0)里

```
class Person{
    string name;
    string phoneno;
    string PID;
public:
    Person(char *s){
        ...
    }
    void print(){
        ...
    }
};
```

```
class Currency;
class Date;
```

对象成员成员的构造顺序：需要补充

```
class SavingAccount{
public:
    SavingAccount(参数表0):saver(...), balance(...), openday(...) { 然后才是这里 }
    ~SavingAccount();
    void print();
private:
    Person saver;      //成员1：其他类Person的对象
    Currency balance;  //成员2：其他类Currency的对象
    Date openday;      //成员3：其他类Date的对象
}

main() {
    SavingAccount sa1(...);
    sa1.print(); return 0;
}
```

//先构造3个子对象，后构造对象自身sa1
//析构相反顺序

注意：初始化列表里为成员变量名；不是类名

初始化列表的参数包含在(参数0)里

```
class Person{
    string name;
    string phoneno;
    string PID;
public:
    Person(char *s){
        ...
    }
    void print(){
        ...
    }
};
```

```
class Currency;
class Date;
```

一个例子:

```
class Employee{
    public:
        Employee(...);
        ~Employee();
        void print();
    private:
        char firstName[100];
        char lastName[100];
        //下面两行应该写什么?
        ??????????
        ??????????
}
```

```
Employee::Employee(char *fname, char *lname,
                    int bmonth, int bday, int byear,
                    int hmonth, int hday, int hyear)
:birthDate(bmonth, bday, byear), hireDate(hmonth, hday, hyear) {
    ???????
    ???????
}
```

```
class Date{
    int year;
    int month;
    int day;
    public:
        Date(int, int, int) {...}
};
```

一个例子:

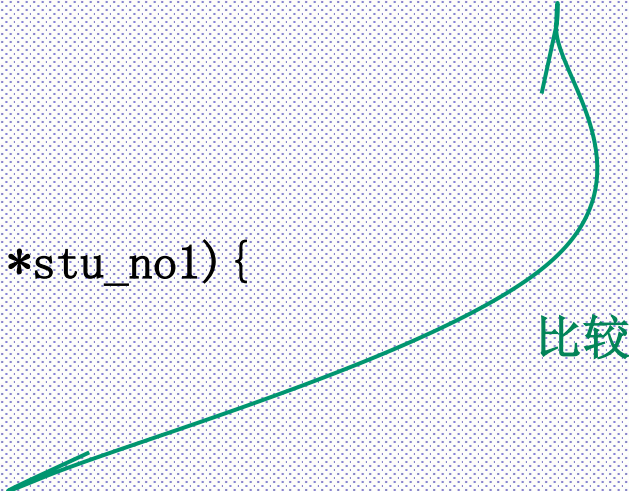
```
class Employee{
    public:
        Employee(...);
        ~Employee();
        void print();
    private:
        char firstName[100];
        char lastName[100];
        //下面两行应该写什么?
        Date birthDate;
        Date hireDate;
}
```

```
Employee::Employee(char *fname, char *lname,
                    int bmonth, int bday, int byear,
                    int hmonth, int hday, int hyear)
:birthDate(bmonth, bday, byear), hireDate(hmonth, hday, hyear) {
    strcpy(firstName, fname);
    strcpy(lastName, lname);
}
```

```
class Date{
    int year;
    int month;
    int day;
    public:
        Date(int, int, int) {...}
};
```

NULL和指针内存泄漏:

```
Student::Student() {  
    name=NULL; //如果无数据, 处理指针为空指针  
    stu_no=NULL;  
}  
  
Student::Student(char *name1, char *stu_no1) {  
    name=new char[strlen(name1)+1];  
    stu_no=new char[strlen(stu_no1)+1];  
    strcpy(name, name1); strcpy(stu_no, stu_no1);  
}  
  
Student::~~Student() {  
    if (name!=NULL) delete []name;  
    if (stu_no!=NULL) delete []stu_no;  
}  
  
void Student::setinfo(char *name1, char *stu_no1) {  
    if(name!=NULL) delete []name;  
    if(stu_no!=NULL) delete []stu_no;  
    //先释放原来的数据和空间;  
    name=new char[strlen(name1)+1];  
    stu_no=new char[strlen(stu_no1)+1];  
    strcpy(name, name1); strcpy(stu_no, stu_no1);  
}
```



比较